

석 사 학 위 논 문

전기차 BMS 보안 강화를 위한 블록체인
기반 통합 아키텍처

김재현

부산대학교 대학원
정보융합공학과

2025년 8월

전기차 BMS 보안 강화를 위한 블록체인 기반 통합 아키텍처

이 논문을 공학석사 학위논문으로 제출함

김재현

부산대학교 대학원

정보융합공학과

지도교수 김호원

김재현의 공학석사 학위논문을 인준함

2025년 06월 27일

위원장	백윤주	인
-----	-----	---

위원	권동현	인
----	-----	---

위원	김호원	인
----	-----	---

<제 목 차 례>

I. 서론	2
1. 연구의 배경 및 필요성	2
2. 논문 기여	4
3. 논문 구성	5
II. 관련 연구	5
1. BMS 아키텍처 및 통신 보안	5
2. 블록체인 기반 데이터 무결성 검증	7
3. 차량 네트워크(CAN/CAN-FD) 보안 기법	7
4. 임베디드 보안 통신 및 OTA 업데이트	8
5. 배터리 여권과 공급망 보안	8
6. 기존 연구와의 비교	9
III. 시스템 개요 및 보안 아키텍처	10
1. 전체 시스템 구성도	10
2. CMU-BMU 간 통신 구조 및 데이터 흐름	11
IV. 보안 요구사항 및 설계 원칙	12
1. 참고 표준	12
2. 위협 모델 정의	13
3. 보안 목표	14
4. 설계 제약 조건	14
5. 보안 설계 원칙	15
V. CMU-BMU 간 보안 통신 프로토콜	16
1. 전체 통신 시나리오 흐름	16
2. 보안 통신 시나리오	16
3. CAN-FD 프레임 구조	18
4. 예외 처리 및 장애 복구 정책	19
5. 하드웨어 및 보안 모듈 설명	20
6. 보안 프로토콜 구현	21
VI. 블록체인 기반 데이터 신뢰 구조	32
2. 제3자 검증 및 외부 접근 절차	34

3. 기술 통합의 효과	35
4. 시스템 구현	37
VII. 실험 결과	41
1. 실시간 처리 성능 분석	41
2. CMU 수 증가에 따른 EDDSA 서명 지연시간 측정	43
3. 메시지 보안성 테스트	43
4. 실차 환경 적용 가능성 평가	44
VIII. 결론 및 향후 연구	45
1. 연구 요약	45
2. 기술적/산업적 기여	46

<표 차례>

표 1 기존 연구 비교	9
표 2 CAN-FD 프레임 구성	19
표 3 구성 요소별 구현 기술 및 플랫폼	37
표 4 단계별 처리 지연 측정	42
표 5 예외 조건에 따른 대응 정책	44
표 6 공격 유형에 따른 시스템 반응 및 대응	44

<그림 차례>

그림 1. 전체 시스템 구성도	11
그림 2. CMU-BMU 간 보안 통신 흐름도	12
그림 3. 위협 모델 정의	13
그림 4. CMU-BMU 간 보안 통신 프로토콜 상호작용 도식	16
그림 5 S32K3, S32K1 시리즈 스펙 - NXP	21
그림 6 BMS Agent 내부 처리 흐름 및 모듈 간 관계	33
그림 7 VC 기반 인증 및 PDP/PEP 흐름	34
그림 8 기술 통합 개념도	36
그림 9 Indy Network 구조	38
그림 10 Fabric Network 구조	38

전기차 BMS 보안 강화를 위한 블록체인 기반 통합 아키텍처

김재현

부산대학교 대학원 정보융합공학과

요약

본 논문은 전기차 배터리 관리 시스템(BMS)의 데이터 신뢰성과 통신 보안성을 확보하기 위한 블록체인 기반 임베디드 보안 아키텍처를 제안한다. 기존 BMS는 센서 데이터를 실시간으로 수집하여 차량 제어 및 상태 모니터링에 활용하지만, CMU-BMU 간 통신 구간의 보안 취약성으로 인해 위변조, 스푸핑, 재전송 공격에 노출되어 왔다. 특히 배터리 여권(Battery Passport)과 같은 정책이 본격화됨에 따라, BMS 데이터의 무결성과 출처 증명이 산업적·법적 필수 요건으로 부상하고 있다. 이를 해결하기 위해 본 연구는 AES-128 기반 대칭키 프로토콜과 CMAC 인증을 활용한 CMU-BMU 간 보안 통신 구조를 설계하고, BMU에서 수집된 데이터를 DID로 서명하여 Hyperledger Fabric에 기록하는 제3자 검증 가능한 무결성 구조를 구현하였다. 또한 실시간성과 신뢰성을 모두 만족하기 위해 블록체인과 클라우드 기반 이중 저장 구조 및 정책 기반 접근 제어 체계를 적용하였다. 제안 시스템은 NXP 차량용 MCU(S32K144, S32K3X4EVB-T172) 기반으로 구현되었으며, Simulink 시뮬레이션 및 실차 수준의 펌웨어 실험을 통해 공격 탐지 개념 검증, 최대 25개 CMU 운용 시 처리 시간 확보 등 실시간성과 보안성을 입증하였다. 본 연구는 차량 보안 규제(UN R155, ISO/SAE 21434) 대응은 물론, 배터리 여권, 정비 이력, 진단 플랫폼 등 다양한 응용 분야에 적용 가능한 기술적 기반을 마련하였다.

I. 서론

1. 연구의 배경 및 필요성

전기자동차(Electric Vehicle, EV)의 급속한 보급과 더불어, 배터리 기술은 자동차 산업의 경쟁력을 좌우하는 핵심 요소로 자리 잡고 있다. 특히 배터리 관리 시스템(Battery Management System, BMS)은 리튬이온 기반 고전압 배터리의 전압, 전류, 온도, 충·방전 상태 등을 실시간으로 감시하고, 이를 통해 과충전, 과방전, 발열 등의 위험 상황을 예방하며, 운전자 및 차량 시스템에 최적화된 운용 정보를 제공한다. BMS의 데이터는 단순한 전력 정보에 그치지 않고, 차량 수명 주기 전반에 걸쳐 차량 성능을 평가하고, 배터리의 상태를 인증하며, 중고 거래 및 배터리 재사용 가능성 판단, 심지어 에너지 거래 플랫폼과의 연동까지 가능한 미래 디지털 자산의 핵심으로 부상하고 있다. 따라서 이와 같은 데이터를 어떻게 신뢰할 수 있는 방식으로 수집하고, 외부에 안전하게 전달할 것인가는 향후 전기차 데이터 산업의 기반을 좌우하는 핵심 기술 이슈라 할 수 있다.

특히 최근 유럽연합(EU)을 중심으로 논의되고 있는 ‘배터리 여권(Battery Passport)’ 제도는 이러한 배터리 상태 데이터를 단순 저장이 아닌 신뢰 기반의 디지털 이력 관리 체계로 발전시키고 있다. 배터리 여권은 배터리의 생산, 사용, 유지관리, 열화 상태, 탄소배출량 등의 정보를 디지털화하고, 이를 제3자가 검증할 수 있는 방식으로 공유함으로써, 투명한 가치 사슬과 지속 가능한 순환 경제를 구현하는 것이 목적이다. 이는 향후 유럽 및 글로벌 시장에서 배터리 제조사, 자동차 OEM, 재활용 업체 등에게 법적·기술적 이행 요구사항으로 작용할 가능성이 높으며, 결과적으로 BMS 데이터의 신뢰성, 무결성, 검증 가능성이 국제 인증 수준의 기술 요소로 요구되는 배경이 된다.

일반적인 BMS는 다수의 CMU(Cell Monitoring Unit)과 중앙 통제 유닛인 BMU(Battery Management Unit)로 구성된다. CMU는 개별 셀의 상태 정보를 수집하고, BMU는 이 데이터를 통합하여 차량 내부 제어기 혹은 외부 시스템에 전달한다. CMU와 BMU 간의 데이터 전송은 대부분 CAN 혹은 CAN-FD 통신 프로토콜을 통해 이루어지며, 실시간성과 신뢰성을 기반으로 한다. 그러나 CAN 프로토콜은 설계 당시 보안 위협을 고려하지 않았기 때문에, 오늘날과 같이 차량이 외부 네트워크 및 클라우드 시스템과 연결되는 환경에서는 사이버 보안 위협에 취약한 구조를 보인다. 특히 CMU-BMU 간의 데이터가 인증이나 암호화 없이 송수신되는 경우, 정상 메시지를 캡처 후 재송신하는 재전송 공격(Replay Attack), 메시지 필드 변조 혹은 조작된 값의 전송을 통한 위·변조 공격(Modification/Tampering), 합법적

ECU를 사칭하는 신원 위장 공격(Spoofing) 등과 같은 공격에 노출된다.

이러한 위협은 단순히 데이터의 왜곡을 넘어, 차량의 동작 제어 오류, 안전 기능 무력화, 화재 및 고장 유발 등 물리적 위험으로 직결될 수 있기 때문에, 각국은 자동차 사이버보안에 대한 규제를 강화하고 있다. 대표적으로 유엔 유럽경제위원회(UNECE)의 UN R155 및 UN R156은 차량의 사이버 보안 위협 식별, 탐지, 대응, 복구를 위한 보안 기능을 설계 단계부터 요구하고 있으며, 차량 제조사는 이를 준수하지 않으면 일부 지역에서 차량을 판매할 수 없다. 이러한 환경에서 BMS는 단순한 전력 관리 시스템이 아니라, 신뢰 기반의 데이터 플랫폼이자 보안 위협의 진입 지점으로 간주되고 있으며, 이에 따라 BMS 내부 통신과 외부 데이터 검증 체계 모두에 대한 보안 강화가 요구된다.

그러나 현재 BMS 시스템을 구성하는 CMU와 BMU는 제한된 하드웨어 리소스를 가진 ECU로, 고복잡도의 암호화 연산이나 블록체인과의 직접 연동이 불가능하다. 특히 CMU는 배터리 상태 수집에 최적화된 센서 기반 마이크로컨트롤러로, 비대칭키 연산이나 블록체인 트랜잭션 서명과 같은 고부하 작업은 수행할 수 없다. 따라서 본 연구는 보안 처리의 책임과 신뢰 주체를 BMU에 집중시키는 구조를 제안한다. 즉, 각 CMU는 데이터를 생성하지만, 블록체인에 등록할 신뢰 가능한 데이터 패키지를 구성하고 서명하는 역할은 BMU가 전담한다. BMU는 자체적으로 DID(Decentralized Identifier) 기반의 개인키를 통해 수집 데이터를 서명하고, 해당 서명 정보와 함께 블록체인 네트워크에 기록한다. 제3자는 BMU의 DID Document를 Hyperledger Indy에서 조회하고, BMU의 공개키로 서명을 검증함으로써 데이터가 위조되지 않았으며 신뢰할 수 있는 BMU로부터 생성되었음을 확인할 수 있다.

하지만 이 구조는 BMU 이후의 데이터 위변조 방지는 가능하게 하지만, 여전히 BMU가 수집한 원천 데이터가 실제 신뢰할 수 있는 CMU로부터 온 것인지 여부는 보장하지 못한다. 즉, BMU가 조작된 데이터를 그대로 서명하고 제출할 경우, 블록체인에 저장된 데이터도 조작될 수 있다. 이 문제를 해결하기 위해, 본 연구는 CMU-BMU 간 통신 보안을 강화하는 AES-128 대칭키 기반 인증 프로토콜을 제안한다. 구체적으로, 차량의 시동 시점에서 각 CMU는 BMU에게 세션 키를 유도하기 위한 변수들을 암호화해서 전송하며, 이후 CMU는 해당 세션 키를 이용해 AES-CMAC 기반 메시지 인증코드(MAC)를 생성하고, CAN-FD 프레임의 Payload 내에 이 MAC과 Freshness Counter 값을 함께 삽입하여 보안 메시지를 전송한다. BMU는 이를 수신한 뒤, 카운터 역행 여부를 확인하고, 자체 계산한 MAC과 비교하여 메시지 위·변조 여부를 실시간으로 판별한다. 이러한 방식은 CAN-FD가 허용하는 최대 Payload 크기(64 byte)를 고려하여, CMAC 중 상위

16 byte만 선택적으로 사용하여 보안성과 데이터 효율성을 트레이드 오프한 결과다.

해당 구조의 구현은 차량용 마이크로컨트롤러 S32K144와 S32K344에 각각 내장된 CSEc, HSE 모듈을 활용하여 보안 연산을 하드웨어적으로 가속화함으로써 실시간 처리 성능을 확보하였으며, 블록체인 상에서는 Hyperledger Fabric을 기반으로 실제 데이터를 기록하고, Hyperledger Indy를 통해 DID 기반 신원 검증 절차를 연동하였다. 이를 통해 본 연구는 차량 내부의 실시간 통신 환경과 외부의 데이터 검증 환경을 하나의 아키텍처로 통합한 사례로서, 차량 보안, 데이터 무결성, 블록체인 기술을 융합한 실용적 프레임워크를 제시한다.

2. 논문 기여

본 연구의 주요 기여는 다음과 같다.

첫째, 자원 제약이 큰 ECU 환경에서도 적용 가능한 AES-128 대칭키 기반 보안 통신 구조를 제안하였으며, 이를 통해 재전송 공격, 위변조, 스푸핑 등의 위협을 방지하였다.

둘째, BMU에 DID를 적용하고 블록체인에 데이터를 기록함으로써, 배터리 데이터의 무결성과 신뢰성을 제3자가 검증 가능한 구조로 전환하였다.

셋째, Hyperledger Fabric, Hyperledger Indy를 통합한 블록체인 환경 구성과 함께, 차량용 마이크로컴에서의 CMAC/EDDSA 알고리즘 구현을 통해 실차 수준의 임베디드 보안 구조를 구현하였다.

넷째, Simulink 기반 시뮬레이션과 S32DS 환경에서의 펌웨어 구현을 통해 제안 기법의 타당성과 성능을 실험적으로 입증하였다.

마지막으로 본 연구는 향후 자동차 보안 규제 대응은 물론, 배터리 데이터 기반 진단·거래 플랫폼 확장의 기술적 기반으로 작용할 수 있다.

3. 논문 구성

이 논문은 총 8장으로 구성되어 있으며, 2장에서는 기존 BMS 통신 구조, 블록체인 기술 및 임베디드 보안 기술의 관련 연구를 다룬다. 3장에서는 전체 시스템 구성과 보안 경계, 데이터 흐름을 제시하고, 4장에서는 BMS 환경의 위협 모델과 보안 요구사항을 분석하고 설계 원칙을 설명한다. 5장에서는 BMU-CMU 간의 보안 통신 프로토콜을 설명하고, 6장에서는 블록체인 기반 BMS 구조의 설계 방안을 제안한다. 7장에서는 Simulink를 활용한 실험 환경과 결과를 제시하고 실험 결과를 통해 보안성, 처리 성능, 자원 소모 측면에서 제안 기법을 평가하고, 8장에서 결론과 함께 향후 연구 방향을 제안한다.

II. 관련 연구

1. BMS 아키텍처 및 통신 보안

전기차 배터리 관리 시스템(BMS)은 안전운용을 위해 사이버보안이 필수적인 요소로 대두되고 있다. Murlidharan[1] 등은 위협 모델링을 통해 BMS 취약점을 체계적으로 분석하고 다층 방어전략을 제시하였다. 특히 무선 BMS(wBMS) 도입 시 새로이 나타나는 통신교란 위협까지 고려한 점이 특징이다. Basic[2] 등은 NFC 기반 무선진단 기법을 통해 2차 사용 배터리의 데이터 접근 문제를 해결하였고, Mayer[3] 등은 BMS를 차량에서 에너지저장장치로 전환하는 과정의 보안 공백을 지적하였다. Naseri[4] 등의 설문연구에서는 클라우드 연계 BMS(CBMS)의 등장으로 원격통신 구간 보안과 프라이버시 보호가 핵심 이슈임을 강조하고 있다. 본 연구는 이러한 BMS 보안 연구의 연장선상에서, 실시간 BMS 데이터의 무결성 보장과 안전한 통신 아키텍처를 제공하는 새로운 기법을 제안한다. 기존 연구들이 개념 제시나 위협 식별에 그친 반면, 본 연구는 프로토타입 구현 및 실험 평가를 통해 제안 기법의 실효성을 검증함으로써 실용적 기여를 도모한다.

2. 블록체인 기반 데이터 무결성 검증

블록체인 기술은 데이터 위변조 방지와 투명성으로 인해 차량 데이터 관리에 활발히 도입되고 있다. Chinnaperumal[5] 등은 EV 배터리의 P2P 에너지 거래에 블록체인을 접목하여 충전 데이터 무결성과 탈중앙 전력거래를 구현하였다. Xing[6] 등은 배터리 재활용 공급망에 블록체인을 적용해 추적성 강화와 참여자 인센티브 변화를 분석하였다. Andreica[7] 등은 차량 IDS 경보를 블록체인 원장에 기록하여 ISO 21434 준수형 로그 무결성을 보장하는 방안을 시연했다. El Fellah[8] 등의 연구는 차량 V2V 통신과 공급망 데이터 교환에 블록체인을 도입하는 청사진을 제시하였다. 본 연구는 블록체인의 이러한 강점을 BMS 데이터 무결성 검증에 응용한다는 점에서 차별화된다. 구체적으로, 배터리 상태 정보와 진단 로그를 허가형 블록체인에 저장하여 배터리 수명주기 전체에 걸친 데이터 신뢰성을 확보한다. 이는 기존 작업들이 개념 검증에 머물렀던 것을 넘어, BMS 데이터를 위한 전문화된 블록체인 구조를 설계하여 실시간 임베디드 환경에 적용했다는 데 의미가 있다.

3. 차량 네트워크(CAN/CAN-FD) 보안 기법

CAN 버스 보안은 전통적으로 암호화 부재로 인한 취약점을 다양한 방법으로 보완해 왔다. Rai[9] 등은 딥러닝 기반 IDS로 CAN 트래픽의 비정상 패턴을 탐지하여 암호화 없이도 공격을 차단하는 성과를 보였다. Gao[10] 등은 머신러닝과 통계기법을 조합하여 CAN-FD용 경량 IDS를 구현하였고, Lu[11] 등은 TrinitySec 프레임워크로 CAN-FD 메시지에 인증·암호화를 부가하면서도 실시간 성능을 유지하였다. Xiao[12] 등은 경량 키 분배 프로토콜을 통해 CAN 메시지 인증 구현의 실용화를 앞당겼다. 이러한 연구들에 비해, 본 연구는 BMS와 차량 통신망 간 인터페이스 보안에 초점을 맞추어 종단 간(end-to-end) 데이터 무결성을 보장하는 솔루션을 제시한다. 특히 BMS 제어 명령/센서 데이터가 CAN(FD) 버스를 통해 게이트웨이를 경유할 때 발생할 수 있는 위협을 모델링하고, 이를 방지하는 암호 프로토콜과 이상징후 탐지 기법을 결합한 통합적인 보안 계층을 제안한다. 이는 BMS 도메인과 차량 도메인 간 경계 보호라는 새로운 시도를 한 것으로, 기존 CAN 보안 기법들과 상호보완적으로 작용하여 차량 전체의 사이버 방어력을 향상시킨다.

4. 임베디드 보안 통신 및 OTA 업데이트

차량 ECU의 임베디드 보안은 하드웨어 자원 제약 속에서 구현되어야 한다는 어려움이 있다. Xie[13] 등은 보안 기능 도입에 따른 하드웨어 비용을 최적화하는 알고리즘으로 이러한 문제에 접근하였고, Gehrer[14] 등은 전력 측정 기반 IDS를 통해 ECU 내부 침입 탐지라는 새로운 가능성을 열었다. Kovacevic[15] 등은 OTA 업데이트 과정에 블록체인과 분산 ID를 도입해 펌웨어 무결성 검증을 구현함으로써, 차량 사이버보안의 또 다른 축인 지속적 업데이트 보안을 다루었다. 본 연구 역시 임베디드 환경 제약을 고려하여 경량 보안 통신 프로토콜을 설계하고, OTA를 통한 BMS 펌웨어 인증 방안을 포함함으로써 실용성을 높였다. 예를 들어, 하드웨어 보안 모듈(HSM)을 활용해 암호연산 부하를 줄이고, DID 기반 장치 인증으로 OTA 업데이트의 신뢰성을 담보하는 등 다층적 접근을 취하였다. 이를 통해 본 연구는 임베디드 보안과 블록체인 보안을 아우르는 포괄적인 BMS 사이버보안 프레임워크를 구현하며, 기존 문헌에서 개별적으로 다루어진 기법들을 통합적 시스템 수준에서 구성해낸 점이 차별화된다.

5. 배터리 여권과 공급망 보안

배터리 여권은 EU의 새로운 규제로, 배터리의 전 주기 데이터를 담아 투명한 거래와 재활용을 지원하려는 개념이다. Terkes[16] 등은 배터리 여권의 개념과 활용처를 정리하고 구현 상 과제를 논의하였고, Li[17] 등은 산업 현장의 데이터 요구를 조사하여 배터리 여권 설계에 현실적 인사이트를 제공하였다. Xing[18] 등의 연구는 배터리 여권 실현에 필요한 추적성 기술로서 블록체인의 경제적 효과를 분석하였다. 본 연구는 배터리 여권 맥락에서 데이터 무결성 및 신뢰성을 보강하는 데 주안점을 둔다. 구체적으로 블록체인과 암호기술을 통해 배터리 여권에 포함된 성능·안전 데이터의 진본성을 확인하고, BMS로부터 여권 시스템으로의 데이터 연계를 안전하게 관리하는 방안을 제시한다. 이는 배터리 여권이 제 기능을 하기 위한 기술적 구현 요소를 채워넣는 작업으로서, 정책 개념을 기술 실체로 구체화했다는 점에서 기존 연구들과 차별화된다. 본 연구의 기여는 이러한 여러 분야의 선행연구를 종합하고 한 걸음 더 나아가, BMS 보안과 데이터 무결성 확보를 위한 블록체인 기반 임베디드 솔루션을 실제 동작 가능한 형태로 제시함으로써, 학술 연구의 실용화 및 표준화에 중요한 밑거름을 제공할 것으로 기대된다.

6. 기존 연구와의 비교

논문	사용 기술	배터리 데이터 사용	임베디드 구현	블록체인 사용	배터리 여권 연계 가능성	규제 대응 가능성
김재현 (본 논문)	AES-CMAC, DID, Hyperledger Fabric, S32K	O	O	O	O	O
Murlidhara n et al. (2025)[1]	STRIDE 위협 모델링, ISO 21434 분석	△	X	X	X	O
Basic et al. (2022)[2]	NFC 기반 무선진단, 대칭키 암호	O	△	X	X	X
Mayer et al. (2024)[3]	공격 트리, 보안 위험 시나리오 분석	O	X	X	△	O

Naseri et al. (2023)[4]	클라우드 기반 BMS 보안 분석	O	X	X	△	△
Chinnaperu mal et al. (2025)[5]	EV-P2P 에너지 거래, 블록체인	O	X	O	△	X
Xing et al. (2024)[6]	배터리 재활용 추적성 + 블록체인	O	X	O	O	△
Andreica et al. (2024)[7]	블록체인 기반 CAN IDS 로그 기록	△	X	O	X	O

표 1 기존 연구 비교

III. 시스템 개요 및 보안 아키텍처

1. 전체 시스템 구성도

본 장에서는 전기차 배터리 관리 시스템(BMS)에 적용되는 블록체인 기반 보안 구조의 전체 개요를 기술한다. 제안하는 시스템은 데이터 수집(CMU), 데이터 검증 및 서명(BMU), 데이터 저장 및 공유(블록체인 및 DID 인프라)의 세 계층으로 구성되며, 각 계층은 고유한 보안 역할을 수행한다.

CMU는 개별 배터리 셀의 전압, 전류, 온도 등의 상태 데이터를 주기적으로 측정하는 임베디드 센서 유닛이다. 각 CMU는 NXP S32K344 차량용 마이크로컨트롤러를 기반으로 구현하며, CAN-FD 통신을 통해 BMU로 데이터를 전송한다. 이때 통신 보안성 확보를 위해 메시지 인증 코드(CMAC)를 포함한 보안 메시지를 생성한다.

BMU는 NXP S32K344 차량용 마이크로컨트롤러를 기반으로 구현되며, 내부에 통합된 HSE 모듈을 통해 AES-CMAC, EDDSA 등 보안 연산을 하드웨어 가속으로 수행한다. 이를 통해 실시간 인증 처리, 서명 생성 등의 핵심 보안 기능을 고신뢰 저지연으로 구현할 수 있으며, BMU는 전체 보안 구조의 신뢰 주체(Trust Anchor)로 기능한다. 또한 BMU는 “BMU는 Secure Boot, Debug Lock 등의 기능을 활용하여 자가 무결성을 유지할 수 있다.

블록체인 네트워크(Hyperledger Fabric)는 BMU로부터 수신된 서명 데이터를 불변성 있는 원장에 저장한다. BMU는 자체 DID를 생성하여 데이터를 서명하며, 제3자는

Hyperledger Indy에서 BMU의 공개키를 조회함으로써 서명 검증이 가능하다. 이를 통해 데이터의 생성 주체가 신뢰할 수 있는 노드임을 확인할 수 있다.

이러한 계층 구조는 리소스 제약 하에 있는 CMU의 부담을 최소화하고, 보안 처리를 연산 자원이 풍부한 BMU에 집중시키는 전략을 취한다. 또한, 실시간 통신 기반의 차량 내 환경과 블록체인 기반의 외부 검증 환경을 하나의 통합 보안 프레임워크로 구성함으로써 향후 배터리 여권, 진단 플랫폼, 에너지 거래 시스템과의 연계 기반을 제공한다.

아래 도식은 CMU-BMU-블록체인으로 이어지는 3계층 보안 구조를 시각적으로 표현한다. 각 계층의 주요 역할과 데이터 흐름, 암호 처리 위치, DID 기반 서명·검증 구조가 포함되며, 전기차 내부와 외부 간 보안 경계를 명확히 구분한다.

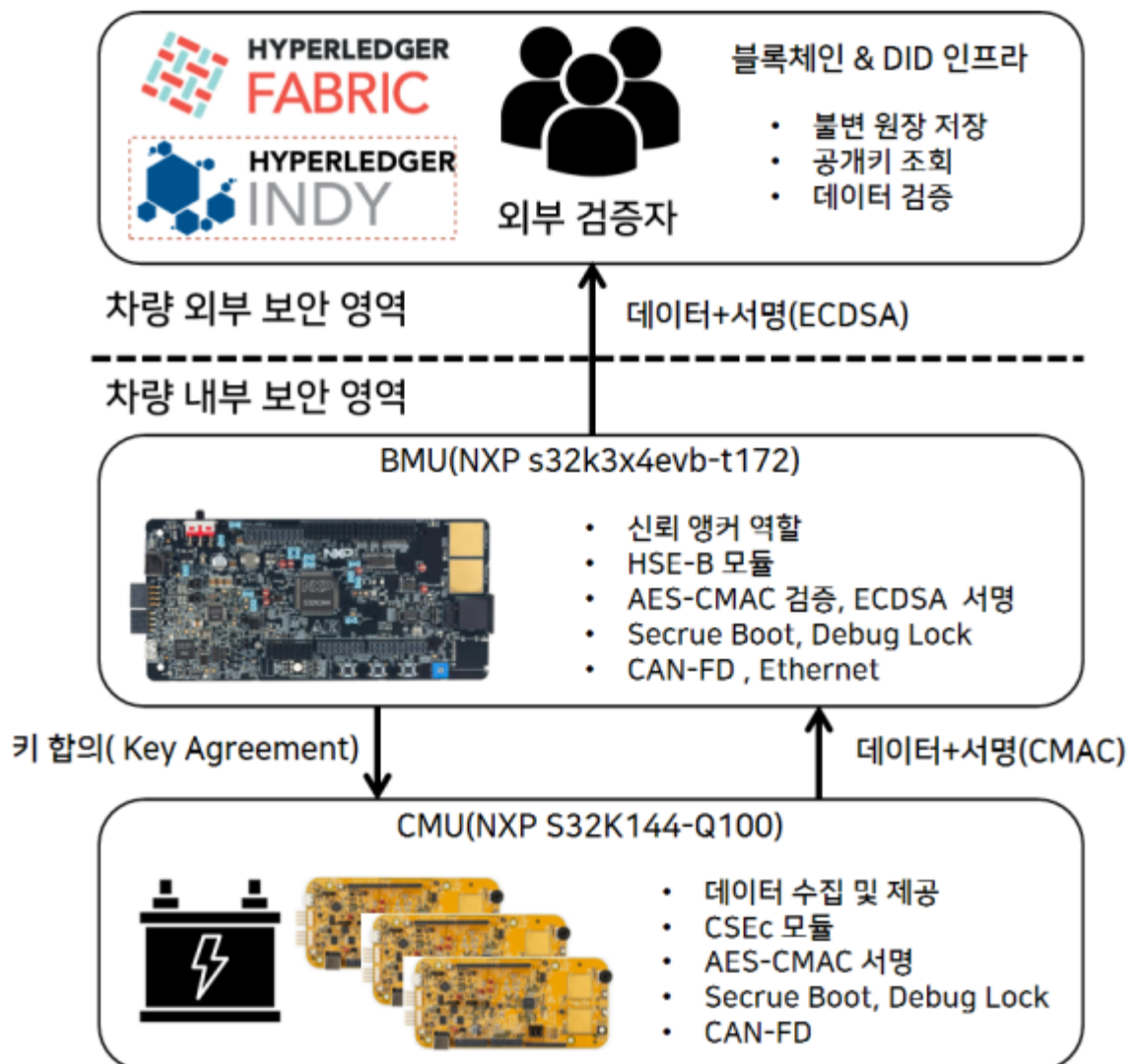


그림 1. 전체 시스템 구성도

2. CMU-BMU 간 통신 구조 및 데이터 흐름

CMU와 BMU 간의 통신은 고속성과 신뢰성이 요구되는 CAN-FD 기반으로 이루어진다. 그러나 CAN-FD는 메시지 암호화, 송신자 인증 등의 보안 기능이 기본적으로 결여되어 있어, 악의적인 재전송, 위조, 신원 사칭 공격에 취약하다. 이를 해결하기 위해 본 연구에서는 사전 공유된 대칭키(PSK)를 기반으로 한 보안 통신 구조를 설계하였다.

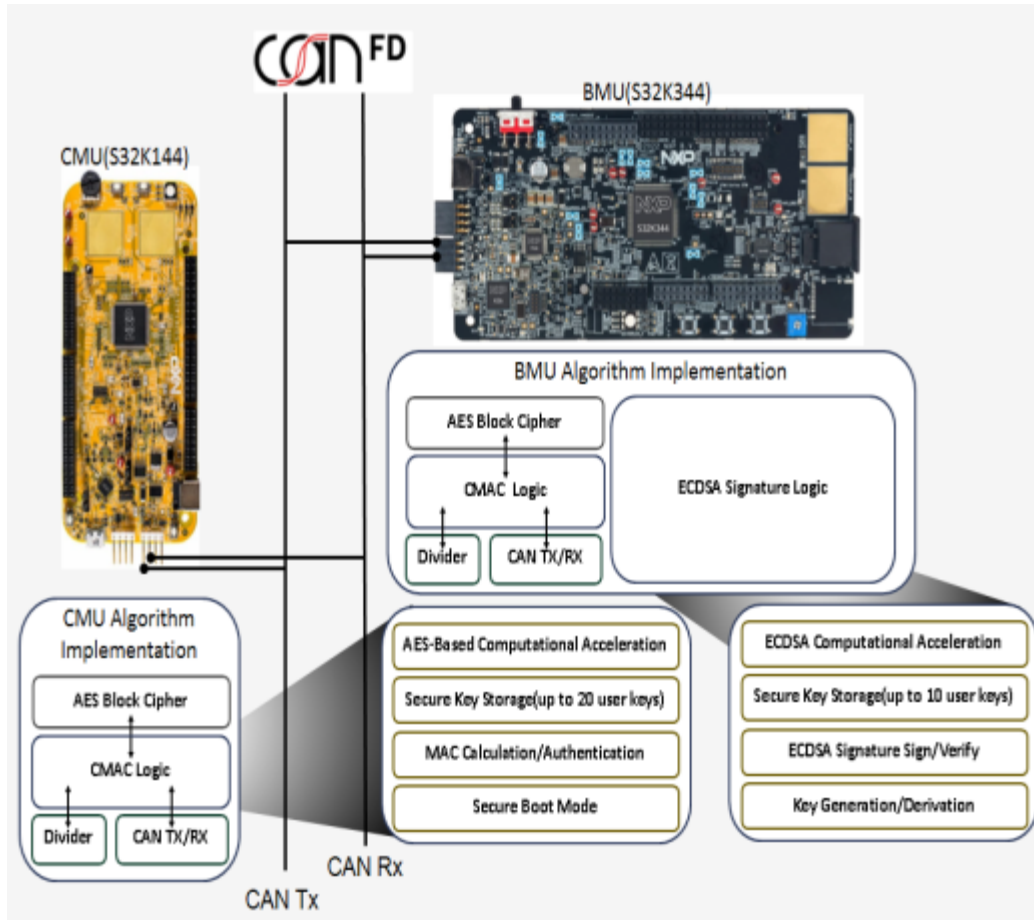


그림 2. CMU-BMU 간 보안 통신 흐름도

본 도식은 전기차 배터리 관리 시스템(BMS) 내에서 CMU와 BMU간의 보안 통신 및 암호 알고리즘 구현 구조를 시각화한 도식이다. 특히 CAN-FD 통신을 기반으로 메시지 인증, 서명, 키 저장 등의 기능이 어떻게 구현되는지를 보여준다.

IV. 보안 요구사항 및 설계 원칙

본 장에서는 차량용 배터리 관리 시스템(BMS)의 사이버 보안 요구사항을 정의하고, ISO/SAE 21434 및 시스템 제약 조건을 반영하여 통신 보안 구조 설계의 원칙을 정립한다.

1. 참고 표준

- ISO/SAE 21434:2021 - Road vehicles - Cybersecurity engineering
- NIST SP 800-108 - Recommendation for Key Derivation Using Pseudorandom Functions
- AUTOSAR SecOC - Secure Onboard Communication
- UN R155 - Cybersecurity and Cybersecurity Management System

2. 위협 모델 정의

차량용 BMS는 배터리 상태 정보를 수집·전송·저장하는 복합적인 사이버-물리 시스템으로, 통신 구간 및 처리 노드에서 다양한 보안 위협에 노출된다. 본 연구에서는 시스템의 논리적 경계를 기준으로 내부 통신 경계(CMU-BMU)와 외부 데이터 연계 경계(BMU-Blockchain)로 구분하고, 각 경계에서 고려해야 할 주요 위협 시나리오를 다음과 같이 정의한다.

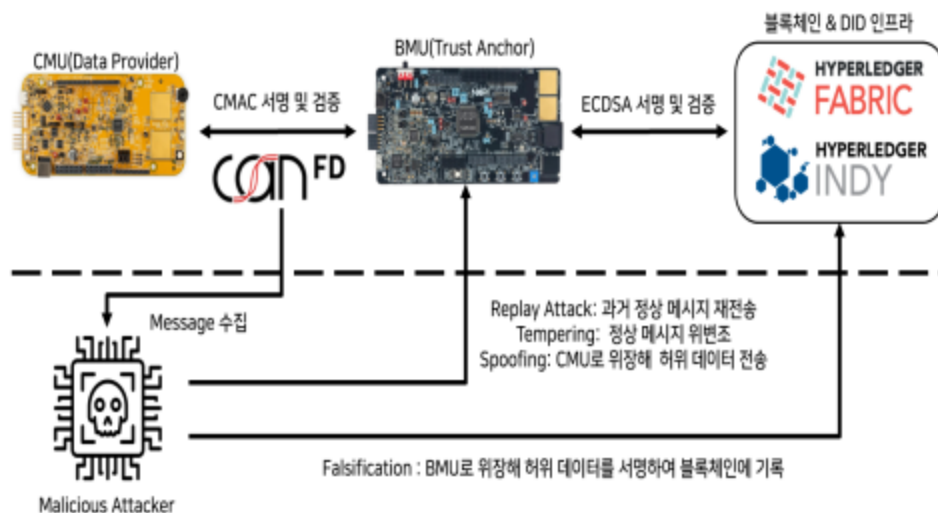


그림 3. 위협 모델 정의

- 재전송 공격(Replay Attack): 공격자가 과거의 정상 데이터를 캡처한 후 동일한 메시지를 반복 전송하여 시스템 상태의 혼란을 유도한다.
- 메시지 위변조(Modification/Tampering): 통신 중 전송되는 배터리 상태 정보 또는 메시지 인증 필드를 변경하여 부정확한 정보를 유포한다.
- 신원 위장(Spoofing): 합법적인 노드(CMU 또는 BMU)의 신원을 사칭하여 허위 데이터를 송신하거나, 무단 접근을 시도한다.
- 데이터 조작(Falsification at BMU): BMU가 악의적으로 변조된 데이터를 블록체인에 서명 및 기록하여 위조된 이력을 생성한다.

이러한 위협은 단순한 정보 유출을 넘어 차량의 안전 제어 기능에 직접적인 악영향을 미칠 수 있으므로, ISO/SAE 21434의 보안 개발 프로세스(Security by Design)에 따라 설계 초기부터 이를 명시적으로 고려해야 한다.

3. 보안 목표

위에서 정의한 위협 모델을 기반으로, 본 시스템이 충족해야 할 보안 목표는 다음과 같다.

- 기밀성(Confidentiality): CMU-BMU 간 통신 구간에서 송신되는 배터리 상태 정보가 외부에 노출되지 않도록 보호한다.
- 무결성(Integrity): 센서 데이터의 생성부터 블록체인 저장에 이르는 전 과정에서 데이터의 위·변조 여부를 탐지 및 방지한다.
- 인증성(Authentication): 모든 메시지의 송신 주체가 합법적인 장치임을 증명할 수 있어야 하며, 메시지 자체의 진본성 또한 확인 가능해야 한다.
- 비부인성(Non-repudiation): BMU가 생성·서명한 데이터를 외부 제3자가 검증 가능해야 하며, 사후에 이를 부인할 수 없어야 한다.

이러한 목표는 ISO/SAE 21434의 요구사항인 ID/Verification, Message Integrity, Origin Authentication과 일치하며, UN R155의 위협 식별 항목과도 정합성을 가진다.

4. 설계 제약 조건

전기차 BMS는 실시간 제어 및 고신뢰성이 요구되는 환경에서 작동하며, 구성요소(CMU, BMU)는 자원 제한적인 임베디드 마이컴(예: S32K144, S32K344)으로 구현된다. 이에 따라 다음의 제약 조건 하에서 보안 구조가 설계되어야 한다.

- 리소스 제약(Limited Resources): CMU는 암호 연산 능력이 극히 제한적이며, 하드웨어 기반 보안 모듈(CSEc)을 활용하여 제한된 범위 내에서 대칭키 연산만 수행 가능하다.
- 실시간성(Real-time Constraint): 모든 보안 처리는 수 밀리초(ms) 단위 이내에 완료되어야 하며, CAN-FD 메시지의 전송 지연이 발생해서는 안 된다.
- 네트워크 제한(Network Efficiency): CAN-FD는 최대 64바이트의 Payload 제한이 있으므로, MAC 및 데이터 필드를 최적으로 압축·배치해야 한다.

5. 보안 설계 원칙

상기 보안 목표와 제약 조건을 종합하여 본 연구에서는 다음과 같은 아키텍처 설계 원칙을 수립하였다.

1) 대칭키 기반 통신 인증 구조

CMU-BMU 간 통신에서는 사전 공유 대칭키(Pre-Shared Key, PSK)를 기반으로 한 세션 키 분배 및 메시지 인증 구조를 채택한다. 시동 시 BMU는 각 CMU에 대해 세션 키에 대한 랜덤한 난수(Seed)를 생성한 후, AES-ECB 방식으로 PSK 기반 암호화를 수행하여 전달한다. CMU는 사전 공유 대칭키로 암호화된 Seed를 수신하고 이를 복호화한다. 복호화된 Seed와 PSK를 사용해서 CMU는 해당 세션 키를 유도한다. 이후 메시지 전송 시마다 Freshness Counter와 배터리 데이터로 AES-CMAC 기반 인증코드를 생성하고, 데이터와 함께 CAN-FD 프레임에 삽입한다. 이는 Replay Attack 및 메시지 변조에 효과적인 대응을 가능케 한다.

2) DID 기반 신원 증명 및 외부 무결성 검증

BMU는 수신한 데이터 패킷을 자체 DID기반 전자서명(EDDSA)을 통해 서명하고, Hyperledger Fabric 블록체인에 기록한다. 제3자는 Hyperledger Indy를 통해 해당

DID Document를 조회하고, BMU의 공개키로 서명을 검증함으로써 데이터의 신뢰성과 생성 주체를 확인할 수 있다. 이 구조는 BMU에 대한 신뢰를 기반으로 외부 검증 환경을 구성한다.

3) 계층적 보안 책임 분산 구조

보안 연산 부담을 자원 제약이 덜한 BMU로 집중시키고, CMU는 데이터 측정 및 인 증코드 생성을 담당하는 경량 역할만 수행한다. 이로써 실시간성과 보안성을 양립할 수 있는 아키텍처가 구현된다. 또한, 블록체인은 데이터의 불변성 및 비부인성을 확보하는 최종 검증 계층으로 기능한다.

V. CMU-BMU 간 보안 통신 프로토콜

본 장에서는 CMU와 BMU간의 보안 통신 프로토콜 구현을 중심으로, FreeRTOS 기반 태스크 구조, CAN-FD 통신 방식, 하드웨어 보안 모듈을 활용한 암호화 및 서명 처리 방식에 대해 구체적으로 기술한다. 해당 구현은 자원 제약이 큰 차량용 ECU 환경에서도 실시간성과 보안성을 동시에 만족시키도록 설계되었다.

1. 전체 통신 시나리오 흐름

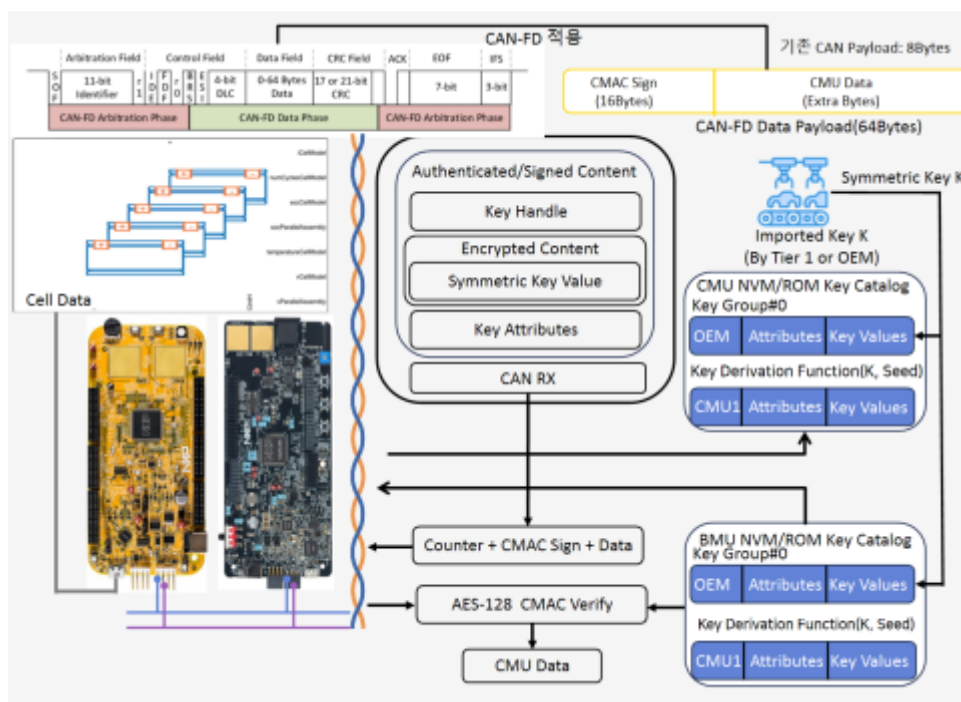


그림 4. CMU-BMU 간 보안 통신 프로토콜 상호작용 도식

위의 도식은 CMU와 BMU 간 보안 통신 프로토콜의 전체 상호작용을 시각적으로 표현한 것이다. 각 단계는 시동 시 세션 키 분배부터 센서 데이터의 전송, MAC 검증, EDDSA 서명에 이르기까지 일련의 과정을 시간 순서에 따라 나타낸다. 색상 구분을 통해 노드 간 역할을 시각적으로 명확히 하고, 데이터 흐름과 보안 연산의 경계를 직관적으로 이해할 수 있도록 설계하였다.

2. 보안 통신 시나리오

본 시스템은 CMU와 BMU간의 보안 통신을 통해 배터리 데이터를 신뢰성 있게 수집하고 처리한다. 통신 초기화 단계에서 세션 키를 생성하고, 이후의 센서 데이터 전송 및 인증 과정까지 전 과정을 하드웨어 기반 보안 모듈(HSE, CSEc)을 활용하여 보호한다.

1) 시동 시 - 세션 키 생성 및 배포

차량 시동 시, 각 CMU_i 는 세션 키 유도를 위한 난수 $Seed_i$ 를 생성한다. 이 값과 고유 식별자 $UUID_i$ 는 사전 공유된 대칭키 PSK_i 를 이용하여 다음과 같이 각각 AES-ECB 방식으로 암호화된다.

- $C_{UUID} = AES_{ECB}(PSK_i, UUID_i)$
- $C_{seed} = AES_{ECB}(PSK_i, Seed_i)$

이후 두 암호문은 결합되어 하나의 CAN-FD 메시지로 구성되고 BMU에 송신된다.

- $M_i = C_{UUID} || C_{seed}$ (총 32Byte)

2) BMU - 세션 키 복호화 및 저장

BMU는 수신한 메시지 M_i 를 분리한 후, 사전 저장된 PSK_i 로 복호화하여 $UUID$ 와 $Seed$ 를 추출한다:

- $\widehat{UUID}_i = AES_{ECB}^1(PSK_i, C_{UUID})$
- $\widehat{Seed}_i = AES_{ECB}^1(PSK_i, C_{Seed})$

BMU는 복호화된 $\widehat{UUID}$ 값을 내부 Whitelist와 비교하여 해당 CMU의 신원을 검증한 뒤, 다음과 같은 KDF를 통해 세션 키를 유도한다:

- $K_{session}^{(i)} = CMAC_{PSK_i}(Label \parallel UUID_i \parallel Seed_i \parallel 0x01)$

Label은 상수 문자열(예: "SessionKey")이며, KDF 입력 구분을 위해 사용되며, Counter는 1회 반복 구성이므로 고정값 0x01을 사용한다.

동일한 연산은 CMU 측에서도 수행되며, 양측은 동일한 128비트 세션 키를 도출하게 된다. 유도된 세션 키는 일반 메모리에 노출되지 않고, HSE 또는 CSEc의 RAM 키 슬롯에 직접 저장된다.

3) CMU - 센서 데이터 측정 및 전송

CMU는 주기적으로 배터리 셀의 전압, 전류, 온도 등을 측정하여 48바이트 크기의 데이터 Payload를 구성한다. 해당 데이터를 보호하기 위해, CMU는 앞서 유도한 세션 키 $K_{session}^{(i)}$ 와 Freshness Counter FC_i 를 기반으로 AES-CMAC을 계산한다

- $MAC_i = CMAC_{K_{session}^{(i)}}(FC_i \parallel Payload)$

최종 전송 메시지는 다음과 같이 구성된다.

- $Message_i = MAC_i \parallel Payload$ (총 64 Byte)

4) BMU - 메시지 수신 및 검증

BMU는 수신된 메시지에서 MAC과 Payload를 분리하고, 로컬에서 동일한 세션 키 및 동기화된 Freshness Counter FC_i 를 기반으로 MAC을 재계산한다.

- $\widehat{MAC}_i = CMAC_{K_{session}^{(i)}}(FC_i \parallel Payload)$

재계산된 MAC과 수신된 MAC이 불일치할 경우, BMU는 메시지를 폐기하거나 재동기화 요청을 수행한다. 모든 통신 노드는 Freshness Counter를 매 전송 시 $FC_i \leftarrow FC_i + 1$ 방식으로 증가시켜, 재전송 및 순서 교란 공격에 대응한다.

5) 보안 효과 및 적용 가능성

본 통신 시나리오는 세션 키 기반의 인증 구조를 통해 메시지의 무결성, 인증성, 재전송 방지를 보장한다. 또한, 세션 키는 매 시동 시 동적으로 유도되며, 하드웨어 보안

모듈(HSE/CSEc)을 통해 안전하게 저장됨으로써 키 유출 가능성을 원천적으로 차단한다. 이와 같은 구조는 실시간성을 요구하는 임베디드 환경에서도 적용 가능하며, 차량용 통신 보안 요구사항(예: ISO/SAE 21434)을 충족한다.

3. CAN-FD 프레임 구조

1) 프레임 구성

제안 구조는 CAN-FD의 식별자(ID) 필드를 통해 송신 노드를 구분하고, Payload는 데이터와 인증 태그만으로 구성되어 통신 효율을 극대화한다.

필드	설명	크기 (Byte)
CAN ID	송신 CMU 식별자 (헤더 필드)	4
Battery Data	전압, 전류, 온도 등 상태값	48
CMAC Tag	AES-CMAC-128 결과 전체 (TAG)	16

표 2 CAN-FD 프레임 구성

CMU-BMU 간 통신은 CAN-FD의 최대 Payload 크기인 64바이트를 활용하여, 센서 데이터(48B)와 CMAC 인증 태그(16B)를 정확히 구성함으로써 데이터 유실 없이 인증 기능을 포함한다. CMU의 식별자는 Payload가 아닌 CAN ID 필드를 통해 전달되어, 데이터 영역을 100% 활용할 수 있도록 하였다.

또한, 향후 위치 정보, SOC 히스토리 등 추가 필드가 필요할 경우를 대비해, CMAC 태그를 8~12바이트로 절단하거나 다중 프레임 연계를 통해 구조적 확장성이 확보되도록 설계하였다.

4. 예외 처리 및 장애 복구 정책

보안 통신 환경에서는 정상 시나리오뿐 아니라 다양한 오류 및 장애 상황을 고려한 설계가 필수적이다. 본 시스템은 다음과 같은 예외 상황을 정의하고, 이에 대응하기 위한 복구 절차를 포함하여 신뢰성과 연속성을 강화하였다.

1) 세션 키 난수 복호화 실패 (CMU 측)

복호화 대상 메시지의 무결성이 손상되었거나 PSK 설정이 일치하지 않을 경우, CSEc는 복호화에 실패한다. 복호화 실패 시 CMU는 세션 키를 초기화하지 않으며, CAN-FD로 'Key Decode Failure' 알림 메시지를 BMU에 전송한다. BMU는 해당 CMU에 대해 암호화된 세션 키 난수를 재전송하며, 재시도 횟수는 3회로 제한한다.

2) MAC 검증 실패 (BMU 측)

BMU가 수신한 메시지에 대해 CMAC 검증을 수행한 결과 MAC 불일치가 발생할 경우, 인증 실패로 간주하고 해당 메시지를 폐기한다. 이와 함께 BMU는 해당 CMU와의 Counter 동기화를 의심하고, 일정 횟수 이상 반복 실패 시 재동기화 메시지를 전송한다. 재동기화 메시지 수신 후 CMU는 세션 키와 Counter를 모두 초기화하여 통신을 재개한다.

3) 태스크 중단 또는 Queue 오버플로우 (RTOS 레벨)

FreeRTOS 기반의 통신 구조에서 태스크 예외가 발생하거나, 큐에 데이터가 과도하게 적재될 경우 데이터 유실 가능성이 존재한다. 각 태스크는 Watchdog Timer와 연동되어 주기적으로 상태를 보고하며, 응답 실패 시 시스템은 안전 모드로 전환된다. 안전 모드에서는 통신을 차단하고, 내부 로깅 및 사용자 알림(예: CAN 진단 메시지 전송)을 수행한다.

4) HSE 또는 CSEc 보안 모듈 오류

암호 연산 중 내부 모듈 오류가 발생할 경우, API 함수는 에러 코드를 반환하며, 해당 오류는 비휘발성 로그에 저장된다. 보안 모듈 오류는 통신 재시도로 해결되지 않으며, 시스템 리셋 또는 안전 모드 진입이 요구된다.

5) DID 서명 실패 또는 전송 오류

BMU의 EDDSA 서명 과정에서 키 손상, 저장소 접근 오류 등이 발생하면 해당 메시지는 블록체인 기록 대상에서 제외된다. 서명 실패 여부는 로그 및 HSE 이벤트로 알림 처리되며, 반복 발생 시 키 재등록 절차가 수행된다. 본 예외 처리 구조는 예상 가능한 다양한 장애 시나리오에 대해 일관된 정책과 복구 흐름을 제공하며, 임베디드 보안 통신 환경에서의 안정성과 실차 적용 가능성을 뒷받침하는 핵심 기반이 된다.

5. 하드웨어 및 보안 모듈 설명

본 연구에서는 NXP의 차량용 MCU에 내장된 보안 모듈을 활용하여 프로토콜을 구현하였다.

1) S32K144 (CMU) - CSEc

- CSEc (Cryptographic Services Engine - compact)는 CMU용 MCU에 내장된 경량 보안 엔진
- 최대 10개의 키 슬롯을 제공, PSK는 NVM에, 세션 키는 RAM 키 슬롯에 탑재
- 지원 연산: AES-128 ECB, AES-CMAC 등 → 실시간 CMAC 생성 가능 (48B 기준 약 1.2ms)
- CMAC 입력은 내부 DMA 방식으로 처리되어 CPU 부하를 최소화하며, 초기화 및 운영은 전용 명령어 셋을 통해 제어됨

2) S32K3X4EVB-T172 (BMU) - HSE

- HSE (High Security Engine-B)는 고성능 보안 엔진으로 BMU용 MCU에 내장
- 수십 개의 키를 NVM/RAM에 저장 가능하며, 접근 권한 제어 및 Secure Boot, Debug Lock 지원
- 지원 연산: AES-ECB, AES-CMAC, EDDSA, ECDSA 등 → 병렬 CMAC 검증 가능 (0.5ms 내 처리)
- HSE는 별도 RTOS 태스크와 연동되며, 인터럽트 기반 처리로 실시간성 확보 가능

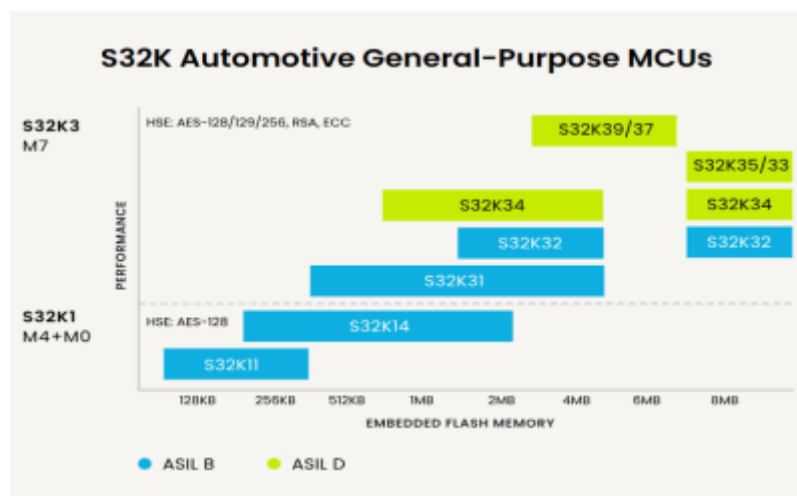


그림 5 S32K3, S32K1 시리즈 스펙 - NXP

6. 보안 프로토콜 구현

본 장에서는 제안한 CMU-BMU 간 보안 통신 프로토콜이 실차 환경에서 어떻게 구현되었는지를 상세히 기술한다. 구현 대상 플랫폼은 NXP S32K144 (CMU 역할) 및 S32K3X4EVB-T172 (BMU 역할)이며, RTOS 기반으로 FreeRTOS를 사용하였고, FlexCAN 및 HSE 보안 API를 통해 실시간 통신 및 암호 연산을 수행하였다.

1) 시스템 구조 개요

제안된 보안 통신 구조는 FreeRTOS 운영체제 하에서 각 유닛(CMU/BMU)에 탑재된 보안 모듈(CSEc/HSE)을 활용하여, AES-128 대칭키 기반의 인증, 세션 키 관리, 그리고 메시지 서명을 실시간으로 수행한다. CMU는 NXP사의 S32K144 MCU 기반으로 구현되며, BMU는 상위 성능의 S32K3X4EVB-T172를 기반으로 한다. 통신은 CAN-FD를 통해 이루어지며, 최대 64바이트 Payload를 효율적으로 활용한다. 이는 배터리 상태 데이터와 인증 정보를 모두 포함하면서도 통신 지연을 최소화할 수 있도록 설계된 구조이다.

2) 1.1.2. FlexCAN 기반 통신 구조

CMU와 BMU 간의 보안 메시지 전송은 CAN-FD(Controller Area Network - Flexible Data Rate) 프로토콜과 NXP의 FlexCAN 모듈을 통해 구현되었다. CAN-FD는 기존 CAN보다 확장된 전송 속도(최대 5~8 Mbps)와 데이터 용량(최대 64바이트 Payload)을 제공함으로써, 인증 정보와 센서 데이터를 단일 프레임 내에 안정적으로 포함할 수 있다.

FlexCAN은 NXP MCU에 내장된 AUTOSAR 호환 CAN 통신 컨트롤러로, 다음과 같은 기능을 제공한다:

- 최대 64개의 Mailbox 지원 (송신/수신 슬롯 설정 가능)
- 하드웨어 수신 필터를 통한 메시지 식별자 기반 필터링
- 우선순위 기반 전송, 버스 충돌 탐지 및 복구
- 수신/송신 인터럽트 지원 및 오류 상태 플래그 관리

본 시스템에서는 Rx FIFO 기능과 인터럽트 기반 수신 처리를 결합하여, 보안 메시지 수신 지연을 최소화하였다. 송신 측은 송신 완료 인터럽트를 이용해 전송 완료 후 바로 다음 메시지를 송신 큐에 적재한다. 모든 메시지 처리는 AUTOSAR OS의 CanIf,

PduR, CanTp 계층과 연동 가능한 구조로 추상화되어 있어 향후 AUTOSAR 통합에도 유연하게 대응 가능하다.

3) FreeRTOS 기반 구조

보안 프로토콜의 실시간성과 태스크 간 처리 병렬성을 확보하기 위해, 본 시스템은 FreeRTOS(Free Real-Time Operating System) 환경에서 태스크 기반 보안 아키텍처를 구현하였다. FreeRTOS는 차량용 MCU 환경에서 요구되는 경량성, 우선순위 기반 스케줄링, 인터럽트 대응 지연 최소화 등의 특성을 제공하며, 보안 연산과 통신 처리의 세분화 및 최적화를 위한 적합한 플랫폼이다.

보안 기능은 다음 세 가지 핵심 태스크로 분할되어 있으며, 각 태스크는 기능적 독립성과 시간적 예측성을 갖춘 구조로 설계되었다:

- CanRxTask (보안 수신 태스크):
 - FlexCAN 수신 인터럽트 발생 시 활성화되며, 하드웨어 Mailbox로부터 CAN-FD 메시지를 읽고, 이를 구조체 형태로 추상화한 후 큐를 통해 후속 처리 태스크로 전달한다.
 - CMAC 인증 태그(최대 16바이트)와 센서 데이터 Payload(48바이트)는 별도로 분리되어 구조체에 저장된다.
 - 메시지 유효성 검사(프레임 ID, 길이 등)를 사전 수행하여 불필요한 처리 낭비를 줄이고, Rx FIFO의 오버플로우 상황에서는 로깅 및 재동기화 요청을 발생시킨다.
- DataProcessTask (인증 및 후처리 태스크):
 - CMU에서는 수신된 메시지의 CMAC를 CSEc를 통해 재계산하여 비교함으로써 위·변조 여부를 실시간 검증한다.
 - BMU에서는 메시지가 유효할 경우, SHA-256 해시 및 EDDSA 서명을 HSE를 통해 실행한다. 이 과정에서 보안 연산을 위한 비동기 요청 큐와 HSM 응답 처리 콜백 구조를 사용한다.
 - CMAC 검증 실패 시 해당 메시지를 폐기하고, 필요한 경우 재동기화 요청 메시지를 전송하도록 Tx 태스크에 플래그를 설정한다.
- CanTxTask (보안 송신 태스크):

- DataProcessTask로부터 인증 또는 서명 완료된 메시지를 수신받아 송신 큐에 적재하고, FlexCAN Mailbox에 할당한다.
- 송신은 인터럽트 기반으로 이루어지며, 전송 성공 시 콜백을 통해 상태 플래그를 업데이트하고, 필요 시 다중 메시지 전송을 처리한다.
- 송신 실패 또는 충돌 시에는 자동 재전송이 수행되며, 재시도 횟수 초과 시 오류 로그를 기록한다.

태스크 간 동기화는 FreeRTOS의 메시지 큐(QueueHandle_t)와 이진 세마포어(Binary Semaphore), 이벤트 그룹(EventGroupHandle_t)을 복합적으로 활용하여 구현되었다. 큐는 포인터 기반 구조체 전달 방식으로 메모리 복사 오버헤드를 줄였고, 세마포어는 공유 자원(CAN 버퍼, 서명 슬롯 등)의 충돌 방지를 위해 사용되었다. 이벤트 그룹은 전역 통신 상태(예: 동기화 완료, 송신 실패 등)를 모듈 간 비동기적으로 공유하는데 활용되었다.

4) CMU 보안 구현: S32K144 + CSEc 기반의 AUTOSAR Crypto 구조

본 연구에서 제안한 CMU-BMU 간 보안 통신 구조는 차량용 마이크로컨트롤러 NXP S32K144에 내장된 경량 하드웨어 보안 모듈인 CSEc(Cryptographic Services Engine - compact)을 기반으로 구현되었다. CSEc는 자동차 보안 표준인 SHE(Secure Hardware Extension)를 따르며, AES-128 기반의 대칭키 암호화, CMAC 생성 및 검증, 난수 생성, 키 저장 및 접근 제어 등 다양한 보안 기능을 하드웨어 수준에서 지원한다.

특히, 해당 보안 기능은 AUTOSAR(AUTomotive Open System ARchitecture) 보안 프레임워크의 Crypto Stack 구조에 맞춰 구현되어 있으며, CSEc 연산은 Crypto Driver 계층에 해당하는 Csec_Ip_* API를 통해 제어된다. 이 구조는 상위 Crypto If(Crypto Interface) 계층과의 표준화된 인터페이스를 제공함으로써, 어플리케이션 코드의 하드웨어 독립성을 높이고, 다양한 MCU 플랫폼 간 이식성을 부분적으로 확보할 수 있도록 구현되어 있다.

이러한 구조 하에서 CMU는 BMU로부터 수신한 세션 키를 PSK(Pre-Shared Key)를 이용하여 복호화하고, 이후 CAN-FD 메시지에 대해 AES-CMAC 기반 메시지 인증을 수행한다. 중요한 보안적 고려사항은, 복호화된 세션 키가 MCU의 일반 메모리에 노출되지 않도록 하는 것이다. 이를 위해 CSEc는 복호화 연산 결과를 일반 RAM이 아닌 전용 RAM 키 슬롯(RAM Key Slot)에 직접 저장하며, 소프트웨어 또는 외부 접근을 원천적으

로 차단한다. 이 구조는 메모리 덤프, 디버거, DMA 등을 통한 키 유출 공격에 대해 효과적인 방어를 제공한다.

(1) CSEc 보안 연산 절차

CSEc 보안 기능을 사용하기 위한 전체 흐름은 다음 세 단계로 구성된다:

- 내부 상태 초기화 및 연산 파라미터 설정

보안 연산을 수행하기 전에 내부 상태 구조체(Csec_Ip_pState)를 초기화하고, 수행할 명령 종류, 키 ID, 데이터 포인터, 데이터 길이 등을 설정한다.

```
Csec_Ip_ClearErrorFlags();
```

```
Csec_Ip_UpdateState(eKeyId, eCmd, pInput, pOutput, u32Length);
```

- PRAM 기반 명령 구성

CSEc는 PRAM(Protected RAM)을 통해 암호 연산 요청을 수신한다. 입력 데이터는 Csec_Ip_WriteCommandBytes()를 통해, 길이 정보는 WriteCommandWord()를 통해, 명령 헤더는 WriteCommandHeader()를 통해 구성된다. 헤더에는 명령 ID, 데이터 전달 방식(COPY/ADDR), 시퀀스 플래그, 키 ID 등이 포함된다.

- 명령 실행 및 결과 처리

명령 실행은 Csec_Ip_StartEncDecECBCmd() 또는 Csec_Ip_StartGenMACCmd() 등을 통해 수행되며, 완료 여부는 내부 플래그(bCmdInProgress)로 확인된다. 결과는 ReadCommandBytes()로 읽고, 상태는 ReadErrorBits()로 확인할 수 있다.

```
while (Csec_Ip_pState->bCmdInProgress) {  
    Csec_Ip_WaitCommandCompletion(timeout, elapsed);  
    Csec_Ip_ContinueEncDecECBCmd();  
}
```

(2) 난수 생성 (Random Number Generation)

128비트 난수는 KDF 또는 인증 연산의 비결정성 확보를 위해 사용되며, Csec_Ip_GenerateRnd() API를 통해 생성된다. 내부 시드 값을 기반으로 생성된 난수는 재현 불가능하며, 보안성이 요구되는 연산에 직접 투입된다.

```
Csec_Ip_GenerateRnd(const Csec_Ip_ReqType* pRequest, uint8* pRnd);
```

(3) AES-128 ECB 암호화

시동 시점에서 BMU는 세션 키를 AES-ECB 방식으로 암호화하여 CMU에 전송한다. CMU는 저장된 PSK를 사용해 이를 복호화하며, 이 과정은 Csec_Ip_EncryptEcb() API를 통해 수행된다. 복호화된 키는 일반 메모리로 복사되지 않고 CSEc의 RAM 키 슬롯에만 저장되며, 직접 읽는 것이 불가능하다.

```
Csec_Ip_EncryptEcb(const Csec_Ip_ReqType* pRequest, Csec_Ip_KeyIdType eKeyId, const
uint8* pPlainText, uint32 u32Length, uint8* pCipherText);
```

(4) AES-CMAC 기반 메시지 인증

측정된 배터리 상태 데이터에 대한 무결성 검증을 위해 CMU는 AES-128 기반 CMAC을 생성한다. 이는 Csec_Ip_GenerateMac() API를 통해 수행되며, 생성된 128비트 CMAC은 데이터와 함께 CAN-FD 프레임에 포함되어 전송된다.

```
Csec_Ip_GenerateMac(const Csec_Ip_ReqType* pRequest, Csec_Ip_KeyIdType eKeyId, const
uint8* pMsg, uint32 u32MsgLen, uint8* pCmac);
```

5) BMU 보안 구현 (S32K3X4EVB-T172 + HSE)

BMU는 고성능 보안 모듈인 HSE(High Security Engine - Class B)를 기반으로, 메시지 무결성 검증, 대칭/비대칭 암호 연산, 전자서명 및 키 관리 기능을 하드웨어 수준에서 수행할 수 있다. HSE는 AUTOSAR Crypto Stack의 Crypto Driver 계층에 해당하는 모듈로, 상위 Crypto If를 통해 통합된 API를 제공함으로써 보안 기능의 모듈화 및 이식 가능성을 확보하고 있다.

BMU는 S32K3X4EVB-T172 MCU에서 실행되며, 호스트 애플리케이션은 Messaging Unit(MU)을 통해 HSE에 요청을 전송한다. 각 요청은 MU 채널 단위로 분리되어 처리되며, 요청-응답 주기 동안 해당 채널은 독립적으로 점유된다. BMU는 이러한 구조를 기반으로 CMU로부터 수신된 메시지를 검증하고, 블록체인에 기록할 데이터에 대해 전자서명을 수행한다.

(1) HSE 서비스 요청 절차의 구조

HSE에 서비스를 요청하기 위한 절차는 일반적으로 다음의 세 단계로 구성된다:

- MU 채널 확보 (Channel Allocation)

서비스 요청은 MU의 각 채널을 통해 수행되므로, 요청 전에 사용 가능한 채널을 확보해야 한다. NXP의 HSE IP 드라이버는 Hse_Ip_GetFreeChannel() API를 통해 유효한 채널을 할당받는다.

```
uint8 u8MuChannel = Hse_Ip_GetFreeChannel(MU0_INSTANCE_U8);
```

유효한 채널이 없는 경우, 요청을 수행할 수 없으며, 채널은 요청-응답 주기 동안 독점적으로 사용된다.

- 서비스 디스크립터 및 요청 구조체 구성

실행할 보안 연산을 지정하는 hseSrvDescriptor_t 구조체와 요청 속성을 정의하는 HseIp_ReqType 구조체를 함께 구성한다. 여기에는 연산 종류(srvId), 암호 알고리즘, 키 핸들, 데이터 포인터, 동기/비동기 처리 방식 등이 포함된다. 모든 주소 포인터는 HSE_PTR_TO_HOST_ADDR() 매크로를 통해 변환되어야 한다.

```
hseSrvDescriptor_t* pDesc = &Hse_aSrvDescriptor[u8MuChannel];  
pDesc->srvId = HSE_SRV_ID_SYM_CIPHER;  
pDesc->hseSrv.symCipherReq.cipherAlgo = HSE_CIPHER_ALGO_AES;  
pDesc->hseSrv.symCipherReq.cipherDir = HSE_CIPHER_DIR_ENCRYPT;  
HseIp_aRequest[u8MuChannel].eReqType = HSE_IP_REQTYPE_SYNC;  
HseIp_aRequest[u8MuChannel].u32Timeout = TIMEOUT_TICKS_U32;
```

- 서비스 요청 전송 (Request Submission)

모든 설정이 완료되면, Hse_Ip_ServiceRequest() API를 호출하여 요청을 MU 채널을 통해 HSE에 전송한다.

```
hseSrvResponse_t result = Hse_Ip_ServiceRequest(  
    MU0_INSTANCE_U8,  
    u8MuChannel,  
    &HseIp_aRequest[u8MuChannel],  
    &Hse_aSrvDescriptor[u8MuChannel]  
);
```

결과는 hseSrvResponse_t 타입으로 반환되며, HSE_SRV_RSP_OK는 요청이 성공적으로 처리되었음을 의미한다. 오류가 발생한 경우, 에러 코드는 키 유효성, 파라미터 오류, 권한 문제 등을 나타낼 수 있다.

(2) AES-128 ECB 복호화

해당 함수는 CMU에게 받은 암호화된 랜덤 시드를 복호화하기 위해 사용된다.

- AES-128 ECB 암호화 시그니처

```
static hseSrvResponse_t App_Aes128DecryptEcb
```

```
(
    hseKeyHandle_t targetKeyHandle,
    const uint8_t *pInput
)
```

- targetKeyHandle: HSE에 등록된 AES 키의 핸들
- pInput: 16바이트 암호문 입력
- encryptionType: ENCRYPT_SYNC 또는 ENCRYPT_ASYNC_IRQ 사용

- AES-128 ECB 복호화 요청 구성

```
pHseSrvDescriptor->srvId = HSE_SRV_ID_SYM_CIPHER;
```

```
pHseEncrypt->accessMode = HSE_ACCESS_MODE_ONE_PASS;
```

```
pHseEncrypt->sgtOption = HSE_SGT_OPTION_NONE;
```

```
pHseEncrypt->cipherAlgo = HSE_CIPHER_ALGO_AES;
```

```
pHseEncrypt->cipherDir = HSE_CIPHER_DIR_DECRYPT;
```

```
pHseEncrypt->cipherBlockMode = HSE_CIPHER_BLOCK_MODE_ECB;
```

```
pHseEncrypt->inputLength = 16U;
```

```
pHseEncrypt->pInput = HSE_PTR_TO_HOST_ADDR(pInput);
```

```
pHseEncrypt->keyHandle = targetKeyHandle;
```

- AES 복호화이며, ECB 모드, 16바이트 입력 고정
- 입력 데이터(pInput)와 사용할 키(keyHandle)를 설정

(3) AES-128 CMAC 검증

해당 함수는 수신된 메시지와 CMAC 태그가 일치하는지 확인(검증)하기 위해 사용된다. 이를 통해 메시지의 무결성을 검증하거나 인증된 노드의 요청인지 확인할 수 있다.

- AES-128 CMAC 검증 함수 시그니처

```
hseSrvResponse_t App_Aes128CmacVerify(
```

```
    hseKeyHandle_t targetKeyHandle,
```

```

const uint8_t *pInput,
uint32_t inputLength,
const uint8_t *pTag
);

```

- targetKeyHandle: HSE에 등록된 AES-128 키의 핸들
- pInput: CMAC 검증 대상이 되는 입력 메시지
- inputLength: 입력 메시지의 바이트 길이
- pTag: 검증하고자 하는 16바이트 CMAC 태그

• CMAC 검증 요청 구성

```

pHseSrvDescriptor->srvId          = HSE_SRV_ID_MAC;
pHseMacSrv->authDir                = HSE_AUTH_DIR_VERIFY;
pHseMacSrv->accessMode              = HSE_ACCESS_MODE_ONE_PASS;
pHseMacSrv->sgtOption               = HSE_SGT_OPTION_NONE;
pHseMacSrv->macScheme.macAlgo       = HSE_MAC_ALGO_CMAC;
pHseMacSrv->macScheme.sch.cmac.cipherAlgo = HSE_CIPHER_ALGO_AES;
pHseMacSrv->keyHandle               = targetKeyHandle;
pHseMacSrv->inputLength              = inputLength;
pHseMacSrv->pInput                  = HSE_PTR_TO_HOST_ADDR(pInput);
pHseMacSrv->pTag                    = HSE_PTR_TO_HOST_ADDR(pTag);
pHseMacSrv->pTagLength              = HSE_PTR_TO_HOST_ADDR(&u32CmacTagLength);

```

- AES-128 기반 CMAC 알고리즘을 사용하여, 주어진 입력 메시지(pInput)와 태그(pTag)가 일치하는지 검증한다.
- macScheme를 CMAC + AES로 지정하여 MAC 종류와 내부 암호화 알고리즘을 설정한다.
- 태그 길이(pTagLength)는 시스템 설계에서 설명한대로 16바이트로 고정되어 있으며, 검증 기준으로 사용된다.

(4) EDDSA 서명

CMAC 검증에 성공한 데이터에 대해 EDDSA 서명을 수행한다. HSE는 SHA-256 해시를 적용한 후, 내부 EDDSA 키 쌍을 이용하여 64바이트 내의 서명 값을 생성하며, 이 서명은 향후 데이터와 함께 블록체인에 기록하여 메시지의 무결성과 인증을 보장하는 데 사용된다.

- EDDSA 서명 생성 함수 시그니처

```
hseSrvResponse_t App_EddsaSign(
```

```
    hseKeyHandle_t keyHandle,
```

```
    const uint8_t *pInput,
```

```
    uint32_t inputLength,
```

```
    bool_t bInputIsHashed,
```

```
    hseSGTOption_t sgtOption,
```

```
    uint32_t *pRlen,
```

```
    uint8_t *pR,
```

```
    uint32_t *pSlen,
```

```
    uint8_t *pS,
```

```
);
```

- keyHandle: HSE에 등록된 EDDSA 개인 키의 핸들
- pInput: 서명 대상이 되는 입력 메시지 또는 해시 값
- inputLength: 입력 메시지의 길이 (바이트 단위)
- bInputIsHashed: 입력 데이터가 이미 해시된 경우 TRUE, 아니면 FALSE
- sgtOption: SGT 사용 여부 (HSE_SGT_OPTION_NONE등)
- pRlen, pR: 서명 결과 R 값 및 그 길이
- pSlen, pS: 서명 결과 S 값 및 그 길이

- EDDSA 서명 요청 구성

```
pHseSrvDescriptor->srvId = HSE_SRV_ID_SIGN;
```

```
hseSignScheme_t signScheme;
```

```
signScheme.signSch = HSE_SIGN_EDDSA;
```

```

signScheme.sch.eddsa.bHashEddsa = FALSE;

signScheme.sch.eddsa.contextLength = 0;

signScheme.sch.eddsa.pContext = 0;

pHseSrvDescriptor->srvId = HSE_SRV_ID_SIGN;

pSignSrv->accessMode      = HSE_ACCESS_MODE_ONE_PASS;

pSignSrv->authDir         = HSE_AUTH_DIR_GENERATE;

pSignSrv->signScheme = signScheme;

pSignSrv->keyHandle       = keyHandle;

pSignSrv->pInput          = HSE_PTR_TO_HOST_ADDR(pInput);

pSignSrv->inputLength     = inputLength;

pSignSrv->bInputIsHashed = FALSE;

pSignSrv->sgtOption       = HSE_SGT_OPTION_NONE;

pSignSrv->pSignature[0] = HSE_PTR_TO_HOST_ADDR(pR);

pSignSrv->pSignatureLength[0] = HSE_PTR_TO_HOST_ADDR(pRLen);

pSignSrv->pSignature[1] = HSE_PTR_TO_HOST_ADDR(pS);

pSignSrv->pSignatureLength[1] = HSE_PTR_TO_HOST_ADDR(pSLen);

```

- srvId를 HSE_SRV_ID_SIGN로 설정하여 HSE에 서명 생성을 요청한다.
- authDir은 HSE_AUTH_DIR_GENERATE로 설정하여 서명 "생성" 방향을 지정한다.
- signScheme을 EDDSA+ HSE_EC_25519_ED25519로 설정한다.
- keyHandle은 서명에 사용할 EDDSA 개인 키 핸들을 지정한다.
- pInput은 서명 대상 데이터이며, 해시되지 않은 경우 HSE 내부에서 hashAlgo를 사용해 해시된다.
- bInputIsHashed가 TRUE이면 입력이 이미 해시된 것으로 간주하며, False일 경우 내부에서 입력을 해시하여 사용한다.
- 출력 서명은 두 부분(R, S)으로 나뉘며, 각각 별도의 포인터와 길이 포인터를 통해 반환된다.

6) 실용성 및 보안 구조적 의의

본 보안 구조는 하드웨어 보안 모듈을 기반으로 한 이중 계층 인증 구조를 구현하여 CMU-BMU 간 메시지 무결성과 송신자 인증을 보장하며, AUTOSAR 표준 호환을 통해 시스템 이식성과 유지보수성을 확보하였다. 또한, CAN-FD의 고속 전송 특성과 FlexCAN의 하드웨어 필터링 및 인터럽트 기반 처리 기능을 활용하여 보안 메시지의 실시간성을 확보하였다. 그러나 CSEc/HSE의 구조적 제약과 명령 구성의 복잡성, 리소스 한계는 향후 다노드 확장이나 OTA 보안 연동 시 주요 고려 요소로 작용할 것이다. 이에 대한 보완 방안은 8장에서 논의한다.

VI. 블록체인 기반 데이터 신뢰 구조

본 시스템의 설계는 다음의 두 가지 원칙을 중심으로 진행되었다. 첫째, 데이터의 출처와 무결성에 대한 검증 가능성을 확보해야 한다. 둘째, 외부 이해관계자(정비소, 보험사 등)에게 데이터 제공 시 투명한 인증 절차 및 접근제어가 수행되어야 한다. 이를 위해 각 데이터는 수집 시점에 디지털 서명되어 위·변조를 방지하며, 그 서명 주체에 대한 DID 기반의 신원 정보는 블록체인에 등록된다. 이후 수집된 데이터는 해시 처리되어 별도의 블록체인 원장에 저장되며, 원본은 클라우드 데이터베이스에 보관된다. 외부 검증자는 VC를 기반으로 인증 절차를 수행한 뒤, 필요한 데이터를 요청하고 검증할 수 있다.

이러한 구조를 통해, 실시간 데이터에 대한 신뢰 기반 접근, 저장된 기록의 불변성, 분산된 서명 검증 등이 모두 하나의 프레임워크 내에서 구현 가능해진다.

1) 데이터 수집 및 서명: BMU의 역할

데이터 흐름의 출발점은 차량 내 BMU이다. BMU는 전압, 전류, 온도, SoC(State of Charge) 등 다양한 배터리 상태 데이터를 일정 주기로 수집한다. 하지만 단순히 데이터를 수집하는 것만으로는 신뢰성을 확보할 수 없다. 따라서 본 시스템에서는 Trust Anchor로 동작하는 BMU가 수집된 데이터를 디지털 서명하는 구조를 채택하였다. BMU는 개인키로 데이터를 서명하고, 이 서명된 데이터를 JSON 포맷으로 직렬화하여 다음 단계인 BMS Agent로 전송한다.

2) BMS Agent의 통합 처리: DID 조회, 검증, 저장

BMS Agent는 수신된 서명 데이터를 검증하고 처리하는 중앙 처리 엔진이다. 이 모

들은 크게 세 가지 하위 기능을 포함하고 있으며, 이들 간의 유기적 결합을 통해 블록체인 기반의 데이터 신뢰성이 확보된다.

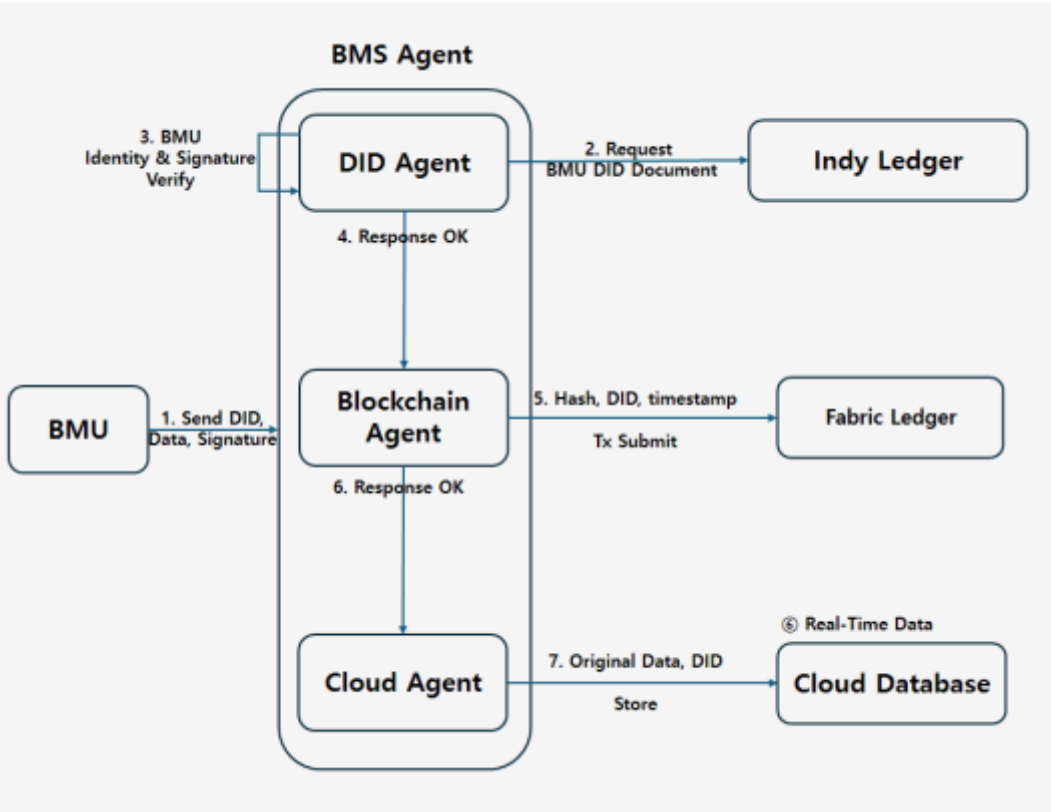


그림 6 BMS Agent 내부 처리 흐름 및 모듈 간 관계

DID Agent는 BMU가 서명 시 사용한 DID를 기반으로, Hyperledger Indy 원장에서 해당 DID 문서를 조회한다. DID 문서는 공개키를 포함한 BMU의 신원 메타데이터를 담고 있으며, 이를 통해 서명의 유효성을 검증할 수 있다. DID 체계는 중앙 집중형 인증 시스템과 달리, 각 주체가 독립적인 신원을 관리할 수 있도록 하며, 제3자 검증 가능성과 프라이버시 보호를 동시에 충족시킨다.

Blockchain Agent는 서명이 유효한 경우, 해당 데이터의 해시를 계산하고 타임스탬프, DID, 서명 등의 메타데이터와 함께 이를 Hyperledger Fabric 블록체인에 기록한다. Hyperledger Fabric은 허가형 블록체인 구조를 갖고 있어, 기업 환경에서의 고성능 처리와 데이터 접근 제어가 가능하다. 기록된 정보는 이후 변경이 불가능하며, 데이터의 출처와 무결성에 대한 영구적인 증거로 활용된다.

Cloud Agent는 해시값만 블록체인에 기록하는 것과 달리, 실제 원본 JSON 데이터를 클라우드 기반의 데이터베이스에 저장한다. 이 설계는 다음과 같은 이유에서 채택되었

다. 일반적으로 블록체인 플랫폼은 높은 보안성과 데이터 불변성이라는 강점을 지니지만, 동시에 낮은 트랜잭션 처리 속도(TPS)라는 구조적 제약을 갖고 있다. 특히 Hyperledger Fabric과 같은 허가형 블록체인의 경우에도, 트랜잭션 검증 및 합의 과정에 따라 지연(latency)이 발생하기 때문에, 고빈도 센서 데이터를 실시간으로 처리하기에는 적절하지 않다.

따라서 본 시스템은 데이터의 신뢰성 검증과 고속 접근이라는 상충되는 요구를 분리하여 처리한다. 블록체인은 데이터의 해시 및 메타정보만을 기록함으로써 위변조 방지 및 출처 검증의 역할을 담당하고, 반면 클라우드 데이터베이스는 원본 데이터를 저장하여 실시간 모니터링이나 대시보드 기반 시각화와 같은 고속 응답 요구를 충족시킨다. 결과적으로, 두 저장소는 기능적으로 분산되지만 논리적으로 연계되어, 실시간성과 신뢰성을 동시에 확보할 수 있는 이중 저장 구조를 구성한다.

2. 제3자 검증 및 외부 접근 절차

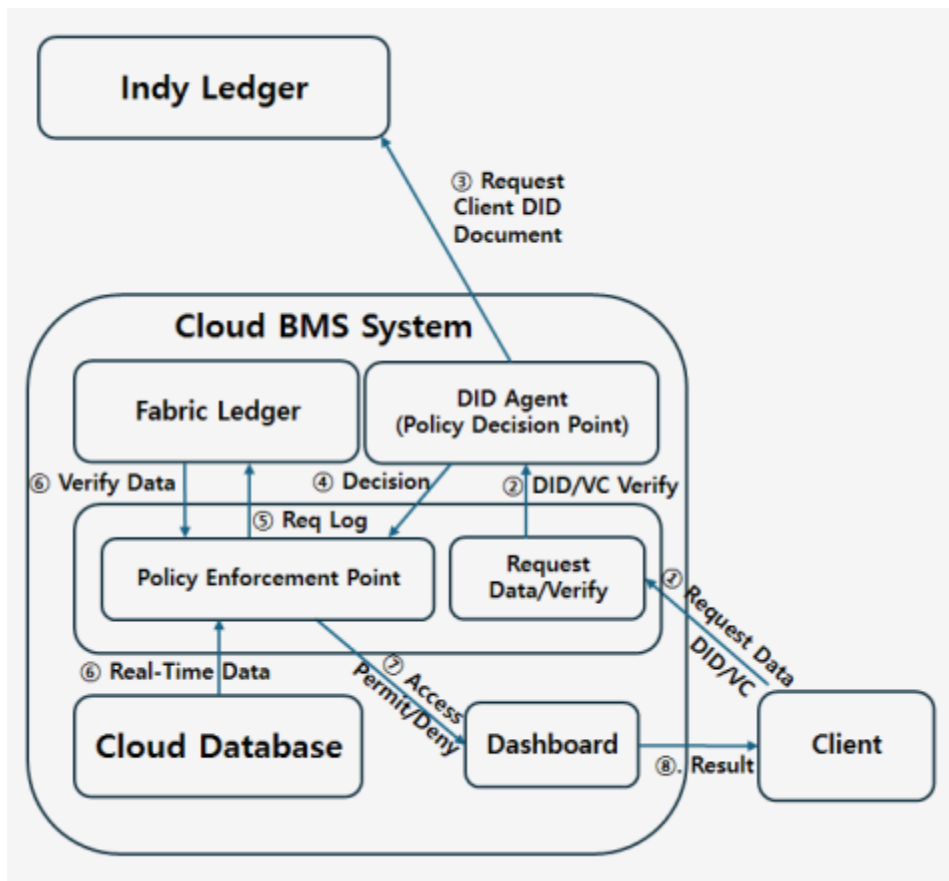


그림 7 VC 기반 인증 및 PDP/PEP 흐름

데이터에 접근하고자 하는 제3자(예: 정비소, 인증 기관 등)는 사전에 발급된 Verifiable Credential(VC)을 이용하여 자신의 신원을 증명한다. 본 시스템은 W3C의 DID/VC 표준을 기반으로 설계되었으며, 이를 통해 제3자의 신원뿐 아니라, 데이터 요청 목적 및 권한 범위까지 함께 평가할 수 있다.

데이터 요청 과정은 접근 제어 시스템의 두 핵심 구성 요소, 즉 정책 결정 지점(Policy Decision Point, PDP)과 정책 집행 지점(Policy Enforcement Point, PEP)을 통해 처리된다. 먼저, 사용자가 VC와 함께 데이터 접근 요청을 전달하면, PEP가 이 요청을 수신하고 필요한 정책 정보를 추출한다. 이후 이 요청은 PDP로 전달되어, 해당 사용자의 VC가 사전에 정의된 접근 정책(예: 데이터 유형, 접근 시간, 역할 기반 조건 등)에 부합하는지를 평가한다. 평가 결과에 따라 PDP는 허용 또는 거부 결정을 반환하며, PEP는 이 결정을 바탕으로 실제 데이터 접근을 허용하거나 차단한다.

이러한 구조는 단순한 인증을 넘어서, 속성 기반 접근 제어(Attribute-Based Access Control, ABAC)의 형태로 확장 가능하다는 장점을 갖는다. 예를 들어, 정비소는 정비 이력 데이터에만 접근할 수 있고, 보험사는 사고 직전의 배터리 상태 정보에만 접근하도록 정책을 정의할 수 있다. 이를 통해 데이터 최소 공개의 원칙(principle of least privilege)이 구현되며, 프라이버시와 보안성이 강화된다.

결과적으로, 본 시스템은 다음의 두 가지 효과를 달성한다. 첫째, 데이터를 요청하는 사용자의 신원과 권한이 엄격히 검증되므로, 무단 접근이나 오용을 방지할 수 있다. 둘째, 수신된 데이터가 실제로 특정 BMU에서 생성되었고, 위·변조되지 않았음을 제3자가 독립적으로 검증할 수 있어, 데이터의 신뢰성과 투명성이 보장된다.

3. 기술 통합의 효과

본 시스템은 단순히 다양한 기술을 조합한 것이 아니라, 데이터 신뢰성 확보라는 목적에 따라 최적화된 기술적 통합 구조를 설계한 데 그 의의가 있다. 각 기술 요소는 독립적으로 강력한 기능을 갖고 있으나, 실제 운용 환경에서는 이들 간의 조화와 역할 분담이 성능과 실효성을 좌우하게 된다. 본 연구에서는 디지털 서명, 탈중앙 신원, 블록체인, 클라우드 저장소, 그리고 접근제어 프레임워크를 유기적으로 결합함으로써, 실시간성과 신뢰성이라는 상충되는 요구를 동시에 만족시키는 구조를 구현하였다.

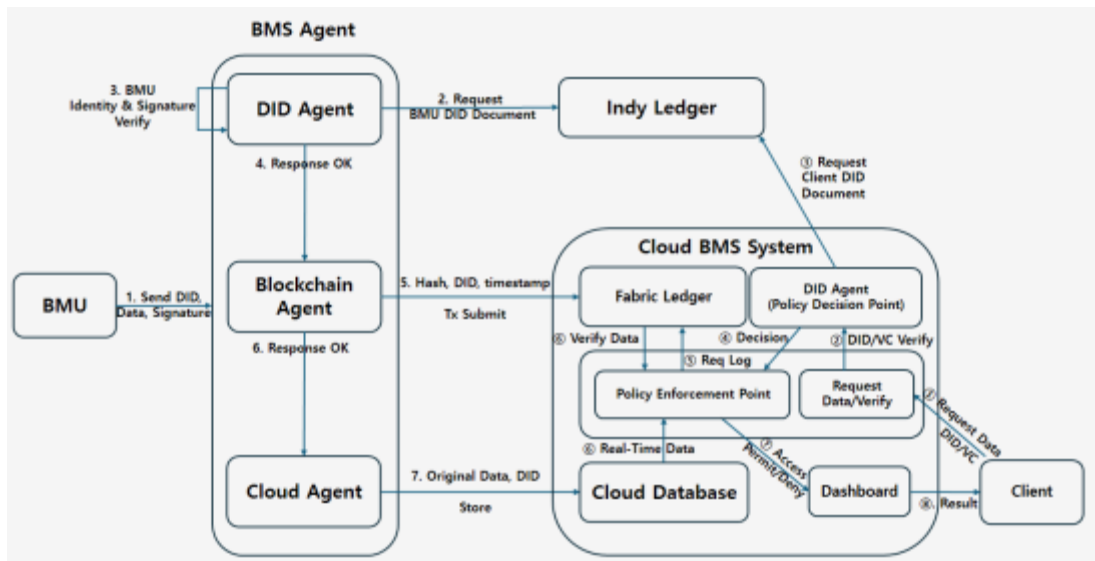


그림 8 기술 통합 개념도

먼저, BMU는 Trust Anchor로서의 역할을 수행하며, 센서 데이터에 대해 EDDSA 기반의 디지털 서명을 생성한다. 이를 통해 데이터의 위·변조 여부는 물론, 해당 데이터가 특정 BMU로부터 생성되었음을 명확히 증명할 수 있다. 서명 주체의 신원은 DID 문서를 통해 공개키 기반으로 검증되며, 이 과정은 블록체인 기반의 Hyperledger Indy를 통해 탈중앙화된 형태로 처리된다. 이는 중앙 기관 없이도 제3자가 서명을 검증할 수 있도록 하며, 데이터의 출처에 대한 검증 가능성과 투명성을 제공한다.

다음으로, Hyperledger Fabric은 데이터의 해시와 메타정보를 저장함으로써 데이터 무결성과 비부인성을 보장한다. 그러나 블록체인의 구조적 특성상 트랜잭션 처리 속도(TPS)가 제한적이기 때문에, 고빈도 센서 데이터를 실시간으로 처리하기에는 적합하지 않다. 이 한계를 보완하기 위해, 원본 데이터는 별도의 클라우드 데이터베이스에 저장되고, 블록체인에는 그에 대한 참조 정보만이 기록된다. 이러한 이중 저장 구조는 실시간 모니터링이 요구되는 클라이언트 요청에 신속히 응답하면서도, 블록체인을 통한 불변성과 추적 가능성을 유지할 수 있도록 한다.

또한, 외부 이해관계자가 데이터를 요청하는 과정에서는 W3C 표준 기반의 Verifiable Credential을 활용한 인증 구조가 도입된다. 해당 구조는 정책 기반 접근 제어 모델과 연계되며, PDP(Policy Decision Point)와 PEP(Policy Enforcement Point) 간 상호작용을 통해 사용자 권한을 동적으로 평가한다. 이를 통해 단순한 신원 확인을 넘어, 역할, 시간, 목적 기반의 세분화된 접근 제어가 가능해지며, 이는 곧 데이터 최소 공개 원칙을 실현하는 기반이 된다.

4. 시스템 구현

제안한 블록체인 기반 데이터 신뢰 구조는 실제 구현을 통해 기능적 타당성과 운용 가능성을 확인하였다. 시스템은 총 6개의 구성 요소(BMU, BMS Agent, Hyperledger Indy, Hyperledger Fabric, Cloud Database, Client)로 이루어져 있으며, 각 구성 요소는 독립적으로 실행 가능하도록 모듈화되어 설계되었다. 본 절에서는 해당 시스템의 구현 환경, 사용 기술 스택, 각 모듈의 구성 방식, 그리고 컴포넌트 간의 연동 방식을 기술한다.

1) 구현 환경 및 기술 스택

시스템 구현은 컨테이너 기반의 마이크로서비스 구조로 이루어졌으며, 주요 기술 스택은 다음과 같다.

구성 요소	구현 기술 및 플랫폼
BMS Agent	Node.js (Express) + Web3 SDK + Indy SDK
DID/VC 처리	Hyperledger Indy (Docker 기반 네트워크)
블록체인 원장	Hyperledger Fabric v2.5 (Docker 기반 네트워크)
Cloud Database	MongoDB (NoSQL 기반 원본 데이터 저장)
PDP/PEP	VC 인증 모듈 (DIDComm 사용)

표 3 구성 요소별 구현 기술 및 플랫폼

2) BMS Agent 구현

BMS Agent는 세 가지 서브 모듈로 분리되어 구현되었다.

- DID Agent는 Hyperledger Indy SDK를 이용하여 BMU의 DID 문서를 Sovrin Test Network에서 조회한다. 이를 통해 BMU의 공개키를 획득하고, 전송된 데이터의 서명 유효성을 검증한다.
- Blockchain Agent는 서명이 검증된 데이터를 해싱하고, Fabric 네트워크의 Chaincode를 통해 해당 해시와 DID, 타임스탬프를 트랜잭션으로 기록한다. Fabric 네트워크는 Docker-Compose를 통해 구성된 1 Organization, 2 Peer 구조로 운영되었다.
- Cloud Agent는 MongoDB를 연결하여 서명된 원본 JSON 데이터를 저장하며, 데이터에 대한 인덱싱과 빠른 검색을 위해 센서 ID, 시간, DID 등을 기준으로 스키마를 구성하였다.

3) Hyperledger Indy 및 Fabric 설정

Indy는 DID 문서 저장을 위해 4개의 노드 validator)로 구성된 프라이빗 네트워크를 사용하고, Indy CLI와 SDK를 활용해 BMU의 DID 등록 및 조회 절차를 자동화하였다.

Indy의 각 노드는 블록체인 합의와 트랜잭션 검증에 참여하며, Plenum 합의 프로토콜을 기반으로 한 RBFT(Redundant Byzantine Fault Tolerance) 알고리즘을 사용하였다.

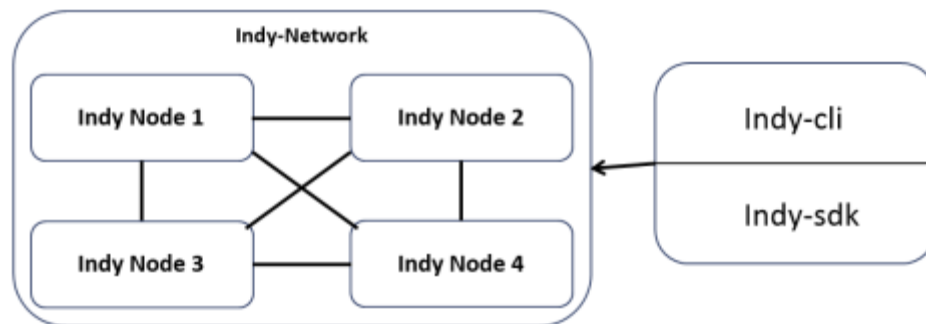


그림 9 Indy Network 구조

Fabric은 블록체인 기반 데이터 무결성 검증을 위해 사용되었으며, Chaincode는 Go 언어로 작성되었다. 트랜잭션 기록에는 데이터 해시, 생성 시각, DID, 검증 결과가 포함된다. 모든 트랜잭션은 CouchDB 기반 상태 DB에 저장되며, 블록으로 구조화되어 오더러 노드에서 처리된다.

Fabric 네트워크는 2개 조직(Org1, Org2) 각 조직은 1개의 피어 노드(peer node) 운영 및 4개의 Ordering Service 노드(Raft 기반)로 구성하였다.

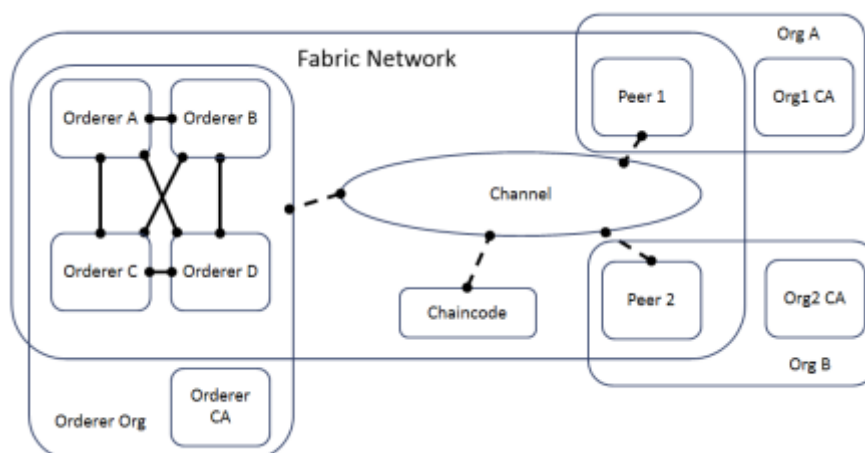


그림 10 Fabric Network 구조

4) 클라이언트 인증 및 접근 제어

클라이언트는 React 기반의 웹 대시보드를 통해 시스템에 접근하며, 이때 Verifiable Credential(VC)을 이용한 DID 기반 분산 신원 인증 절차를 수행한다. 본 시스템은 W3C DID/VC 표준을 기반으로 설계되었으며, 사용자는 사전에 발급된 VC를 제출함으로써 자신의 신원, 역할, 권한 범위를 시스템에 증명할 수 있다.

클라이언트가 인증을 마치고 데이터 접근을 요청하면, 시스템은 정책 집행 지점(Policy Enforcement Point, PEP)을 통해 요청을 수신하고, VC 내 포함된 속성(역할, 목적, 접근 범위 등)을 추출하여 정책 결정 지점(Policy Decision Point, PDP)에 전달한다. PDP는 요청 정보를 사전 정의된 정책과 비교·평가하여, 허용 또는 거부 결정을 반환하며, 이에 따라 PEP는 해당 요청을 실행하거나 차단한다.

데이터 접근 정책은 데이터 민감도 및 사용 목적에 따라 세 가지 수준의 접근 계층(Tier)을 정의함으로써, 보다 세밀한 권한 통제를 가능하게 한다.

- Tier 1: 공개 수준 데이터 (Public Tier)
 - Tier 1 데이터는 배터리의 기본적인 작동 상태 정보로, 민감도가 낮고 비교적 자유롭게 공유 가능한 범주에 해당한다. 예를 들어, 배터리 온도 범위, 일일 충전 횟수, 평균 SoC 수준 등이 포함된다.
 - 접근 주체는 정비소, 사용자 본인, 인증기관 등으로, VC에 역할 정보가 포함되기만 하면 별도의 심층 정책 없이 접근이 허용된다.
- Tier 2: 중간 민감도 데이터 (Restricted Tier)
 - Tier 2 데이터는 사고 직전의 전류/전압 패턴, 급속 충전 이력, 셀 불균형과 같은 정보로, 상황 분석 및 사후 평가에 유의미한 데이터를 포함한다. 이 데이터는 사용자의 동의 혹은 목적 제한을 조건으로 제공된다.
 - 접근 주체는 수리기관, 연구기관 등으로 제한되며, 시간 조건이나 차량 소유자의 동의 여부 등의 추가 조건이 평가된다.
- Tier 3: 고민감도 데이터 (Confidential Tier)
 - Tier 3은 BMU 내 개별 셀 전압의 시간 시리즈, 열화 예측 계수, 제조사 특화 파라미터 등의 기밀 정보 또는 원천 데이터로 구성되며, 외부 접근은 극도로 제한된다.

- 접근 주체는 제조사 또는 배터리 진단을 위임받은 공식 기관으로 한정된다.

이러한 Tier 기반 접근제어 정책은 PDP 내 정책 평가 엔진의 Rule Set으로 구성되며, 클라이언트 요청은 VC 기반 DIDComm 메시지로 암호화되어 전달되며, 정책 위반 시에는 접근이 거부된다.

이 구조는 다음과 같은 보안적·운영상 이점을 제공한다:

- 데이터 최소 공개 원칙 실현: 필요 최소한의 정보만 공유됨
- 역할 기반 + 속성 기반 접근 통제 융합: 정적 권한과 동적 조건을 모두 반영
- 정책 변경 및 확장 용이성: 정책 정의가 독립되어 있으며, 고도화된 평가 로직 가능

결과적으로, 본 시스템은 민감도 계층화와 정책 기반 판단을 결합하여, 다양한 이해관계자에게 데이터 제공 시 투명하고 세밀한 접근 제어 체계를 구현하였다. 이는 실제 산업 응용(정비, 보증, 보험 등)에서 요구되는 보안성과 신뢰성을 충족시키는 핵심 기반으로 작용한다.

5) 시스템 간 연동 구조

전체 시스템은 RESTful API를 통해 각 컴포넌트 간 통신을 수행하며, 주요 데이터 흐름은 다음과 같다.

- BMU → BMS Agent: 센서 데이터 + EDDSA 서명 전송
- BMS Agent → Indy: DID 문서 조회
- BMS Agent → Fabric: 데이터 무결성 해시 트랜잭션 제출
- BMS Agent → MongoDB: 원본 데이터 저장
- Client → BMS Agent: VC 기반 인증 + 데이터 요청
- BMS Agent → Client: 검증된 데이터 응답

이러한 연동 구조는 각 기술이 수행하는 기능을 분리하면서도, 상호 간 데이터 일관성을 유지할 수 있도록 설계되었다.

VII. 실험 결과

전기차의 배터리 관리 시스템(BMS, Battery Management System)은 리튬 이온 배터리의 안전한 운영을 책임지는 핵심 구성 요소로서, 과충전, 과열, 합선 등의 위험을 감지하고 이에 대응한다. 이러한 BMS는 차량의 안전에 직접적인 영향을 미치므로 ISO 26262 자동차 기능안전 표준에 따라 설계되어야 하며, 위험 수준에 따라 자동차 안전 무결성 등급(ASIL, Automotive Safety Integrity Level)이 할당된다.

ASIL C나 ASIL D와 같은 높은 등급의 안전 요구사항이 적용되는 BMS 기능에서는 실시간성이 특히 중요하다. 즉, BMS가 위험을 감지하고 안전 조치를 취하기까지의 레이턴시(latency) 또는 반응 시간(response time)은 해당 등급에서 요구하는 한계 이내여야 한다. 본 연구에서는 ISO 26262 표준과 ASIL 등급 관점에서 BMS의 레이턴시 요구사항과 제약 조건을 기반으로 실험 데이터를 분석하며, 하나의 셀 모니터링 유닛(CMU)이 48바이트 데이터를 전송하고 배터리 관리 유닛(BMU)이 이를 수집하여 서명한 후 BMS로 전송하는 구조에서의 실시간성 요구를 평가한다. 이를 통해 몇 개의 CMU까지 실시간성 요구를 충족할 수 있는지 분석하고, ASIL C/D 수준의 안전 목표를 만족하는 반응 시간 범위를 정량적으로 제시한다.

1. 실시간 처리 성능 분석

전기차 BMS는 고전압 배터리의 안전성과 효율성을 보장하기 위해 센서 데이터를 수집하고 분석한다. 이때 센서 데이터의 측정, 전송, 검증, 저장까지 모든 보안 처리가 해당 주기 내에 완료되어야 한다. 본 연구에서는 AUTOSAR Classic Platform의 RTE Layer 타이밍 명세와 ISO 26262의 제어 루프 응답시간 기준에 따라 100ms 이하를 실시간성 보장 임계값으로 설정하였다.

CMU의 메시지 송신 주기는 50ms로 설정되었으며, 이는 ISO 26262에서 정의한 고장 허용 시간 간격(FTTI)과 ASIL C/D 수준의 안전 목표를 기반으로 한 값이다. 대부분의 BMS 기능은 10~100ms 이내의 반응 시간을 요구하며, 과전압/과온 감지 및 릴레이 차단 등 긴급 조치가 필요한 경우에는 10ms 수준의 응답이 요구된다. TI의 무선 BMS 설계 사례에서는 ASIL D 대응을 위해 모듈 데이터 갱신 주기를 100ms 이내로 설계하고 있으며, 본 논문에서는 실시간성과 안정성을 동시에 확보하기 위해 50ms를 기준으로 설정하였다. 이 주기 설정은 CMU가 각 셀의 전압과 온도 상태를 충분히 반영하면서도 전체 시스템의 응답 지연이 누적되지 않도록 보장한다.

1) 단계별 처리 지연 측정

EDDSA는 비대칭키 기반 디지털 서명 알고리즘으로 보안성과 효율성을 제공하지만 서명 처리 시간이 길어지며 오버헤드가 증가하는 경향이 있다. 본 실험에서는 EDDSA 서명 연산의 지연을 줄이기 위해 각 CMU가 보낸 메시지를 개별적으로 서명하지 않고, 동일한 Freshness Counter 값을 갖는 메시지들을 그룹화하여 서명하는 구조를 채택하였다. 이 방법을 통해 서명 처리 빈도를 줄이고, 메시지 집계 처리 효율을 향상시킬 수 있다.

단계	연산 항목	처리 내용	평균 지연	수행 주체	비고
(1)	CMAC 생성	CSEc 연산으로 CMAC 생성	0.034 ms (34 μ s)	각 CMU	인증 포함 송신 구조 내 완료
(2)	CAN-FD 송수신	FlexCAN 송신 Mailbox 적재 및 전송 인터럽트	무시 가능		통신 지연 무시 가능 수준
(3)	CMAC 검증	HSE를 이용한 수신 메시지 무결성 검증	0.022 ms*10 (0.22ms)	단일 BMU	실시간 수신 처리 구조
(4)	EDDSA 서명	HSE로 서명 수행	20.85ms	단일 BMU	병목 가능 지점
(5)	DID 조회	Indy 네트워크에서 BMU DID 문서 조회	88~120 ms (초기 1회) <1.0ms (캐시 적용)	BMU Agent	DID 공개키 캐시 사용
(6)	서명 검증	Indy 공개키 기반 BMU 서명 검증	0.05 ms	BMU Agent	최적화된 Rust 구현 (ed25519-dalek)

표 4 단계별 처리 지연 측정

2) 결과 분석

전체 보안 경로는 100ms 이내로 처리됨을 실험적으로 확인하였으며, 이는 ISO 26262와ASIL 등급에서 정의하는 실시간 제어 루프 한계 이내의 성능이다. 보안 연산은 CSEc와 HSE 모듈을 통해 하드웨어 가속되며, RTOS 태스크 구조와 인터럽트 핸들링으로 통신과 암호 연산 간 충돌 없이 병렬 처리가 가능함을 보였다.

2. CMU 수 증가에 따른 EDDSA 서명 지연시간 측정

실차 환경을 반영한 시뮬레이션을 통해, CMU 수 증가가 BMU 내 HSE 연산 처리에 미치는 영향을 정량적으로 분석하였다.

1) CMU 수 증가에 따른 지연시간 측정 결과

Simulink 기반 HIL 환경에서 10대 이상의 CMU가 48바이트의 데이터를 주기적으로 전송하고, BMU가 이를 수신하여 디지털 서명 또는 무결성 체크를 수행한 후, 종합 패킷을 BMS 시스템에 전송하는 방식으로 실험을 진행하였다.

- 480 bytes (약 10개 CMU): 53,179 μ s ($\approx$ 53.2 ms)
- 720 bytes (약 15개 CMU): 67,419 μ s ($\approx$ 67.4 ms)
- 960 bytes (약 20개 CMU): 81,659 μ s ($\approx$ 81.7 ms)
- 1200 bytes (약 25개 CMU): 95,899 μ s ($\approx$ 95.9 ms)
- 1296 bytes (약 27개 CMU): 101,595 μ s ($\approx$ 101.6 ms)

실험 결과는 CMU의 수가 증가함에 따라 통신해야 할 데이터 양도 선형적으로 증가하여 지연 시간도 증가함을 보인다.

2) 결과 분석

ISO 26262 기능안전 표준에 따르면 BMS와 같은 전기차의 안전 필수 시스템은 명확한 시간 응답 요구사항을 수립하고 이를 충족하도록 설계되어야 한다. ASIL C/D 등급의 BMS 안전 기능은 배터리 위험으로부터 승객을 보호하기 위해 실시간 동작이 요구된다. 본 연구에서 분석한 결과, 지연 시간은 CMU 수에 비례하여 선형적으로 증가하며, CMU 한 개당 약 2.85ms의 지연이 추가된다. 약 25개 이하의 CMU까지는 실시간성을 유지할 수 있으나, 그 이상에서는 100ms 한계를 초과할 수 있다.

3. 메시지 보안성 테스트

제안된 보안 메시지 구조는 CMU와 BMU 간의 통신에서 발생할 수 있는 다양한 위협 시나리오에 대해 실시간 탐지 및 차단 능력을 제공한다. 이 실험은 ISO/SAE 21434,

AUTOSAR SecOC, UN R155에서 요구하는 보안 요건을 기반으로 설계되었으며, 무결성, 신뢰성, 신선도 보장 측면에서 평가되었다.

1) 예외 처리 및 복구 정책

예외 조건	대응 정책
CMAC 검증 3회 연속 실패	세션 키 초기화 및 최대 3회 재협상
Counter 역행 탐지	재동기화 요청 전송 및 통신 재시작
UUID 인증 실패	메시지 폐기, 디버깅 모드에서는 로그 기록
Key 복호화 실패	'Key Decode Failure' 응답, 키 재전송 요청

표 5 예외 조건에 따른 대응 정책

2) 개념 검증 공격 시나리오

공격 유형	설명	시스템 반응	대응 메커니즘
Replay Attack	이전 정상 메시지를 반복 송신	FC 역행 탐지 → 폐기	FC 기반 시계 검증
Message Tampering	센서 데이터 혹은 CMAC 일부 변조	CMAC 검증 실패 → 폐기	AES-CMAC 재계산 및 비교
Spoofing	미등록 UUID를 통한 위장 전송	UUID 인증 실패 → 폐기	화이트리스트 인증
Counter Guessing	유효한 CMAC 생성을 시도	CMAC 불일치 → 폐기	HSE 기반 키 고유성 검증
MAC Truncation	CMAC 일부 제거 또는 조작	태그 길이 오류/CMAC 불일치 → 폐기	CMAC 길이 + 값 동시 검증
Session Key Injection	암호화된 난수를 조작하여 키 도출 오류 유도	복호화 실패 → 키 초기화 요청	CSEc 복호화 실패 검출

표 6 공격 유형에 따른 시스템 반응 및 대응

4. 실차 환경 적용 가능성 평가

실험 결과를 기반으로, 제안된 블록체인 기반 보안 구조는 실차 환경에 충분히 적용 가능한 성능과 안정성을 확보한 것으로 판단된다. 이를 다음의 세 가지 측면에서 평가하였다

1) 실시간성 (Real-Time Performance)

실시간 처리 요구 조건은 ISO 26262에서 정의하는 제어 루프 응답시간(100ms 이하)을 기준으로 한다. 실험 결과, 20개 이하의 환경에서는 평균 지연 시간이 81.7ms로 실시간성 요구를 여유 있게 만족하며, 15개 이하의 CMU 환경에서는 53.2ms로 매우 안정적인 성능을 보였다. 그러나, 25개 CMU부터 실시간성 요구 기준에 근접하며, 27개 CMU에서는 100ms를 초과해 실시간성 요구를 충족하지 못하는 상황이 발생하였다.

2) 실차 유사성 및 적용 현실성 (Deployment Realism)

실제 전기차의 배터리 관리 시스템은 보통 8~24개의 CMU를 갖추고 있으며, 일부 대형 시스템에서만 30개 이상의 CMU가 필요하다. 제안된 시스템은 30개 CMU까지의 통신량을 충분히 처리할 수 있으며, 실시간 보안 처리를 성공적으로 완료하였다. 이로 인해 상용 EV BMS 아키텍처의 약 90% 이상에 적용 가능함을 보인다.

VIII. 결론 및 향후 연구

본 연구는 전기차 배터리 관리 시스템(BMS)의 데이터 신뢰성과 통신 보안성을 동시에 확보하기 위한 실용적 보안 아키텍처를 제안하고, 이를 임베디드 환경과 블록체인 기반의 외부 검증 시스템에 적용 가능한 형태로 구현하였다. 제한된 연산 자원을 가진 차량용 ECU 환경에서 실시간 성능을 유지하면서도, 재전송 공격, 위·변조, 스푸핑 등의 보안 위협에 대응할 수 있는 통신 프로토콜과, 데이터의 무결성을 제3자가 검증 가능한 블록체인 연계 구조를 설계하였다.

1. 연구 요약

- 보안 통신 구조 설계: CMU-BMU 간 통신에 대해 AES-128 기반 대칭키 인증 프로토콜을 적용하여, 실시간성과 경량성을 확보하였다.
- 신뢰 기반 데이터 구조 구현: BMU에서 수집된 데이터를 DID 기반으로 서명하고 Hyperledger Fabric 블록체인에 기록함으로써, 데이터의 출처성과 무결성을 외부 이해관계자가 독립적으로 검증할 수 있도록 하였다.
- 시스템 구현 및 실험 검증: Simulink 시뮬레이션, 실차 수준 MCU 펌웨어 구현, 블록체인 네트워크 연동을 통해 실시간 성능, 공격에 대한 개념 검증, 시스템 확장성을 실험적으로 검증하였다.

2. 기술적/산업적 기여

본 연구는 다음과 같은 기술적, 산업적 의의를 지닌다:

- 차량 보안 규제 대응: UN R155, ISO/SAE 21434, ISO 26262와 ASIL 등급 등의 국제 자동차 사이버보안 규정에 부합하는 구조로, 향후 전기차 보안 설계 및 인증 대응에 실질적 도움을 줄 수 있다.
- 배터리 여권 기반 기술 확장: 블록체인 기반 데이터 무결성 구조는 EU 중심의 '배터리 여권' 정책에 부합하며, 향후 중고차 거래, 배터리 재사용, 에너지 거래 플랫폼과의 연동에 핵심 인프라로 활용 가능하다.
- 실차 적용 가능성 확보: CSEc/HSE 기반 하드웨어 보안 모듈을 활용하여, 실제 상용 MCU에서 실시간성과 보안성을 동시에 달성한 구조를 제시하였다.

제한점 및 향후 확장 방향

본 연구는 다음과 같은 한계를 지닌다:

- 다수 노드 확장 시 병목: BMU의 HSE 모듈 병목으로 인해 25개 이상의 CMU 확장 시 처리 지연 및 메시지 손실이 유의미하게 증가함을 확인하였다. 향후 HSE Queue 최적화, 서명 주기 간소화, 병렬 처리 구조 개선 등을 통해 처리 용량을 확대할 필요가 있다.
- 서명 데이터 처리 비용: EDDSA 기반 서명 및 블록체인 연동은 높은 신뢰성을 제공하지만, 연산 및 저장 비용이 크다. 이를 개선하기 위한 경량 서명 알고리즘 도입 및 해시 기반 증명 구조(Merkle Tree 등) 적용이 고려될 수 있다.
- OTA 연계 및 보안 업데이트 대응: 본 구조는 정적인 보안 구조에 초점을 맞추고 있으며, 향후에는 무선 펌웨어 업데이트(OTA)와의 연계 및 보안 동적 구성 변경에 대한 연구가 필요하다.

향후 연구에서는 상기 한계점을 개선함과 동시에, 다양한 산업 도메인(정비, 에너지 플랫폼 등)과의 연계를 위한 보안 인터페이스 확장, 정책 기반 데이터 접근 제어 고도화 등을 통해 시스템의 신뢰성과 확장성을 더욱 강화해 나갈 예정이다.

참고문헌

- [1] S. Murlidharan, M. M. Nasr, and K. Kandasamy, "Battery Management System: Threat Modeling, Vulnerability Analysis, and Cybersecurity Strategy," *IEEE Access*, vol. 13, pp. 15523–15540, 2025.
- [2] F. Basic, T. Staake, and C. Witzig, "A Novel Secure NFC-Based Approach for BMS Monitoring and Diagnostic Readout," in *Proc. IEEE RFID Conf.*, 2022.
- [3] K. Mayer et al., "An Analysis of Security Concerns in Transitioning BMS from First to Second Life," in *Proc. ARES Conf.*, 2024.
- [4] F. Naseri, Z. Kazemi, and P. G. Larsen, "Cyber-Physical Cloud Battery Management Systems: Review of Security Aspects," *Batteries*, vol. 9, no. 1, pp. 1–22, 2023.
- [5] S. Chinnaperumal, B. Selvaraj, and D. Ramasamy, "Decentralized Energy Optimization Using Blockchain with Battery Storage and EV Networks," *Scientific Reports*, vol. 15, 2025.
- [6] Q. Xing et al., "Blockchain Technology Embedded in the Power Battery Echelon Recycling under Traceability Mechanism," *Scientific Reports*, vol. 14, no. 1, 2024.
- [7] T. Andreica, A. Petrica, and M. Vochin, "Blockchain Integration for In-Vehicle CAN Bus Intrusion Detection Systems with ISO/SAE 21434 Compliant Reporting," *Scientific Reports*, vol. 14, 2024.
- [8] K. El Fellah et al., "Revolutionizing Automotive Security: Blockchain Solutions for Enhancing Physical Flow in the Automotive Supply Chain," *Computer Systems Science and Engineering*, vol. 45, no. 3, pp. 345–356, 2025.
- [9] R. Rai, M. Jindal, and A. Dey, "Securing the CAN Bus Using Deep Learning for Intrusion Detection in Vehicles," *Scientific Reports*, vol. 15, 2025.
- [10] F. Gao, C. Zhang, and Y. Liu, "Multi-Attack Intrusion Detection for In-Vehicle CAN-FD Messages," *Sensors*, vol. 24, no. 3, 2024.
- [11] R. Lu et al., "TrinitySec: Trinity-Enabled and Lightweight Security

- Framework for CAN-FD Communication," IEEE Trans. Dependable Secure Comput., 2023.
- [12] Y. Xiao, S. Shi, N. Zhang, W. Lou, and Y. T. Hou, "Session Key Distribution Made Practical for CAN and CAN-FD Message Authentication," in Proc. ACSAC, 2020.
 - [13] Y. Xie, J. Chen, and M. Zhu, "Security-Related Hardware Cost Optimization for CAN-FD-Based Automotive CPS," Sensors, vol. 21, no. 12, 2021.
 - [14] S. Gehrer and J. Guajardo, "PHIDIAS: Power Signature Host-Based Intrusion Detection in Automotive Microcontrollers," in Proc. ASV Workshop, ACM CCS, 2024.
 - [15] [A. Kovacevic and N. Gligoric, "Enhancing Security of Automotive OTA Firmware Updates via Decentralized Identifiers and DLT," Electronics, vol. 13, no. 2, 2024.
 - [16] M. Terkes, R. de Graaf, and L. Nijs, "Battery Passport for Second-Life Batteries: Potential Applications and Challenges," IEEE Access, vol. 12, 2024.
 - [17] Y. Li et al., "Data Requirements and Availabilities for a Digital Battery Passport," Sustainable Energy, 2023.
 - [18] X. Xiong et al., "Detection of Cyber Attacks in EV Charging Systems Using a Remaining Useful Life (RUL) Generative Adversarial Network," Scientific Reports, vol. 15, 2025.

Blockchain-Based Integrated Architecture for Enhancing EV BMS Security

JaeHyun Kim

Department of Information Convergence Engineering
The Graduate School
Pusan National University

Abstract

This paper proposes a blockchain-based embedded security architecture for Electric Vehicle (EV) Battery Management Systems (BMS) to address critical data and communication vulnerabilities. With policies like the Battery Passportmaking data integrity a legal necessity, securing the communication link between the Cell Monitoring Unit (CMU) and Battery Management Unit (BMU) against tampering and spoofing is essential. Our solution secures CMU-BMU communication using an AES-128 protocol with CMAC authentication. Data collected by the BMU is signed with a Decentralized Identifier (DID) and recorded on Hyperledger Fabric to ensure verifiable integrity. A dual blockchain-cloud storage architecture is employed to satisfy both real-time performance and reliability requirements. Implemented on NXP automotive MCUs, the system's real-time security was validated through simulations and firmware testing, proving its effectiveness in attack detection and processing capabilities. This research provides a technical foundation for complying with security regulations like UN R155 and ISO/SAE 21434, while also enabling applications such as battery passports, maintenance history, and diagnostic platforms.